Submission - Assignment 1

- Course: Event-driven and Process-oriented Architectures (EDPO), FS2026, University of St.Gallen
- Group 4
  - Evan Martino
  - Marco Birchler
  - Roman Babukh

Repositories

- Project repository: https://github.com/chechmek/EDPO_FS26_Project
- Exercise 1 repository: https://github.com/MrStarco/EDPO_FS26_E1_Kafka_Tests

1. General Project Description

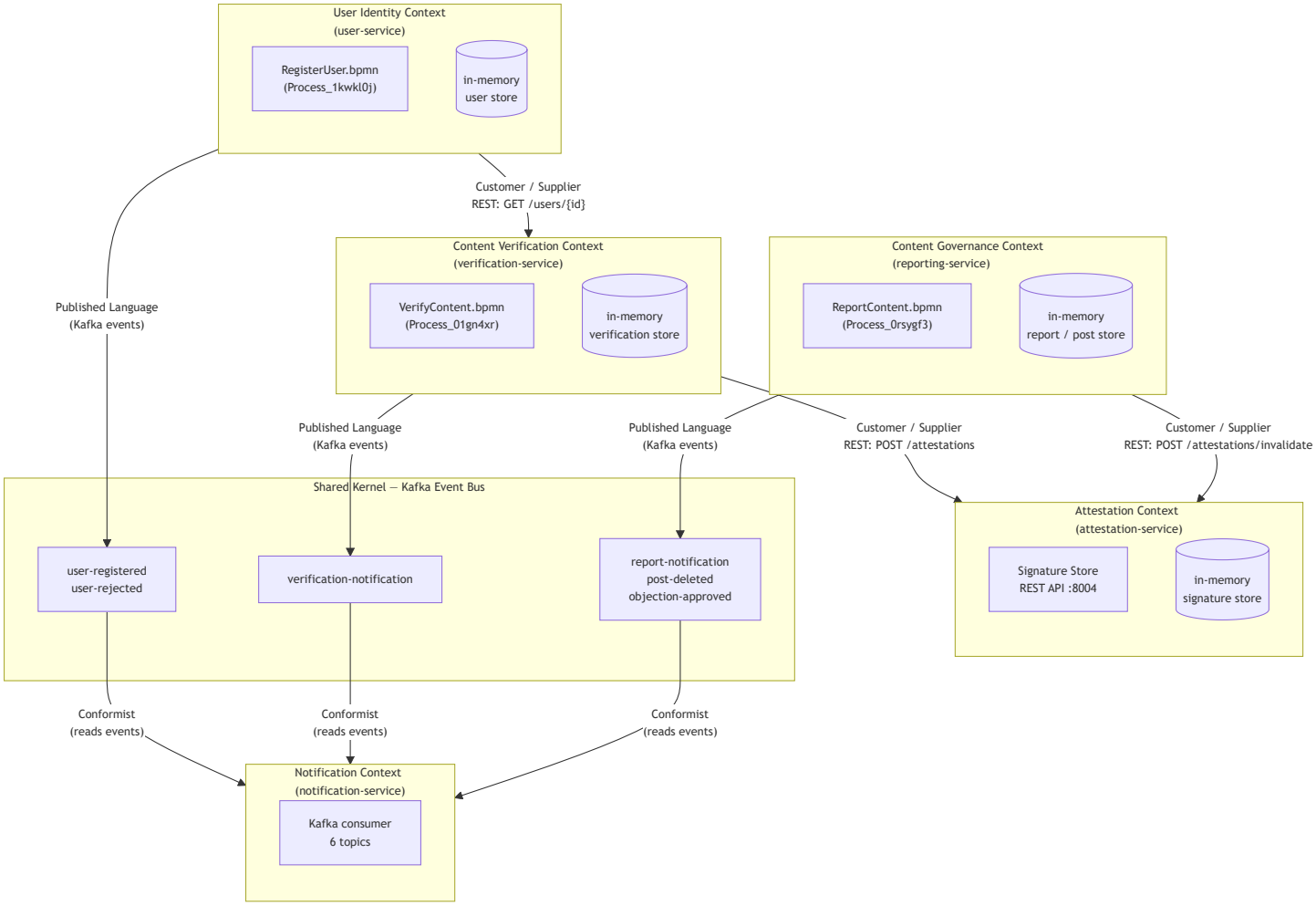
1.1 Project Purpose and Value Proposition

The project is a peer-based content verification platform implemented as a set of Python microservices. Trustworthiness is established through an explicit, auditable process in which a registered user submits content, peers review it, the platform performs additional internal checks, and a cryptographic attestation is created only if the full process reaches a positive outcome. By modelling flows explicitly, the platform can explain why a user was accepted, why content was verified, why a post was deleted, or why an objection was rejected. This information becomes available as part of the attestation signature itself.

1.2 Domain Structure and Bounded Contexts

The system is organised into distinct bounded contexts: the user domain is responsible for onboarding and registration status, the verification domain is responsible for peer-based trust decisions, the reporting domain governs moderation and deletion, the attestation domain stores and maintains signatures, and the notification domain notifies users.

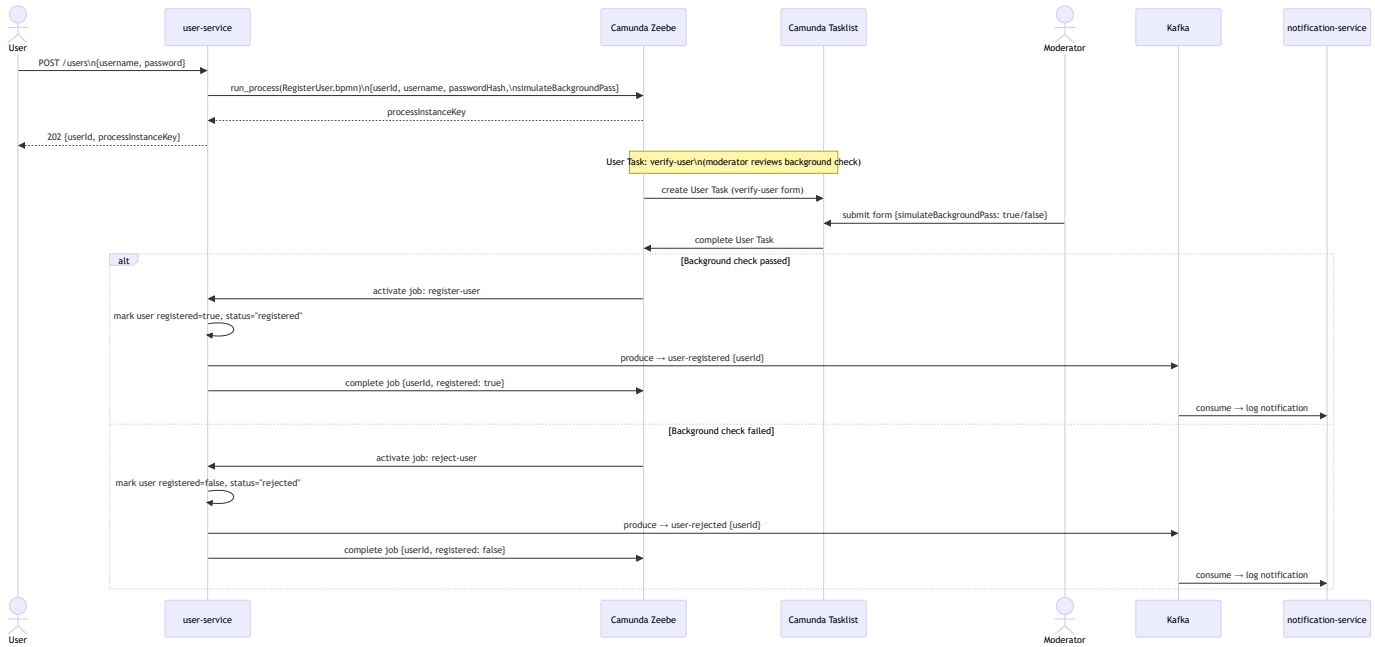
The context map below shows how these domains relate to each other. `User Identity` is upstream of `Content Verification`, because verification depends on registration status. `Attestation` is a reusable capability consumed by both verification and governance. Kafka acts as a shared event bus for notification-style communication, and the `notification-service` behaves as a conformist consumer of the published event language.



1.3 RegisterUser Process

`RegisterUser.bpmn` (`Process_1kwkl0j`) manages user onboarding (see Appendix D for the visual BPMN representation). A registration request is not automatically accepted, because the platform assumes that participation in later trust-related processes should itself be controlled. The flow therefore includes a human-reviewed background check in Camunda Tasklist before the user account is activated. Only after this decision does the service publish either a `user-registered` or a `user-rejected` event to Kafka. This step is also checked in other processes.

The process begins synchronously through the REST API, becomes stateful inside Zeebe, involves a human task, and only then emits an event for downstream consumers.

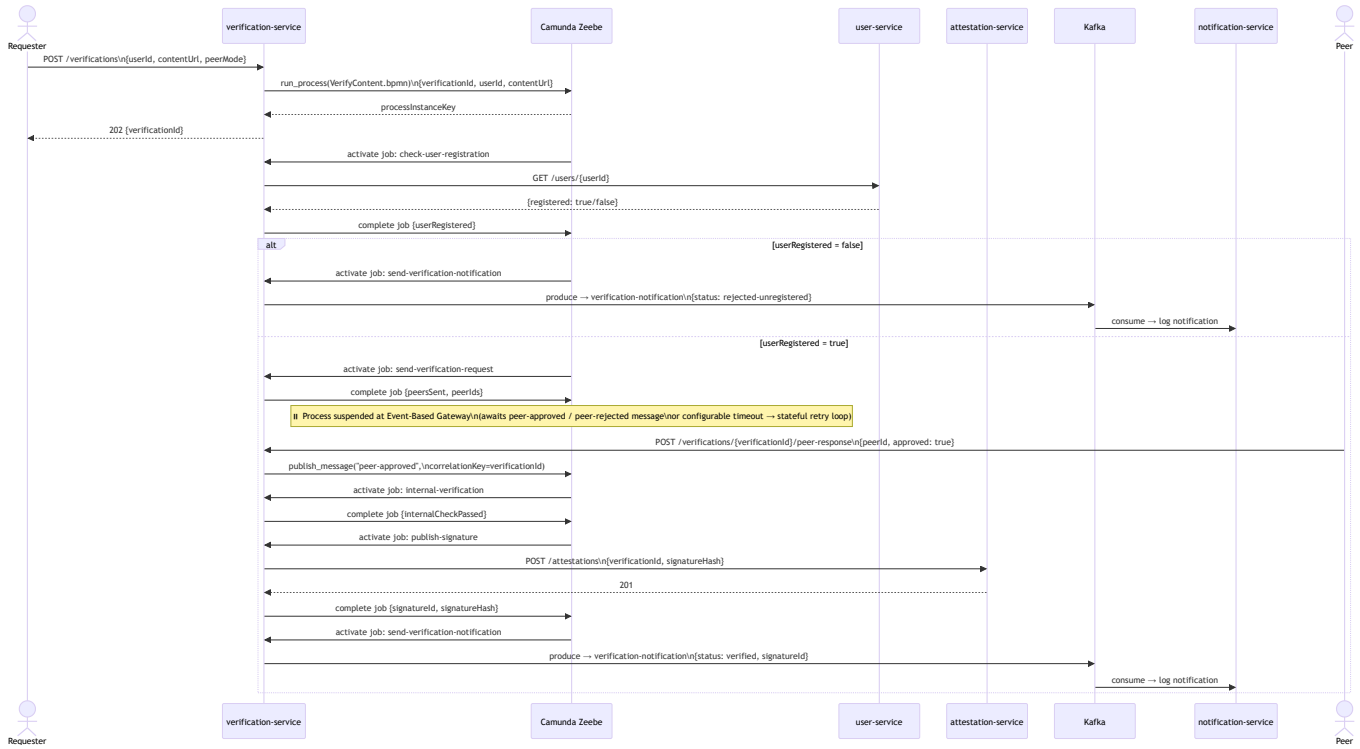


From a technical perspective, this demonstrates the overall style of the platform. The `user-service` exposes the external API, starts the BPMN process, executes the service tasks through a Zeebe worker, and persists its domain-facing state in an in-memory store. The generated `userId` then becomes the stable identifier used by the verification flow.

1.4 VerifyContent Process

`VerifyContent.bpmn` (`Process_01gn4xr`) governs the full content verification lifecycle and is the central process of the platform (see Appendix D for the visual BPMN representation). After a registered user submits content to be verified, the platform checks eligibility, dispatches peer review requests, waits for peer verdicts through message correlation, performs an internal verification step, and, if the outcome is positive, stores a cryptographic signature through the `attestation-service` .

This process again combines several EDPO concepts: synchronous REST to validate a dependency on the `user-service` , asynchronous waiting via an Event-Based Gateway, stateful retry when peer responses are delayed, internal worker-driven service tasks, a REST-based infrastructure call to the `attestation-service` , and Kafka publication of the final result.

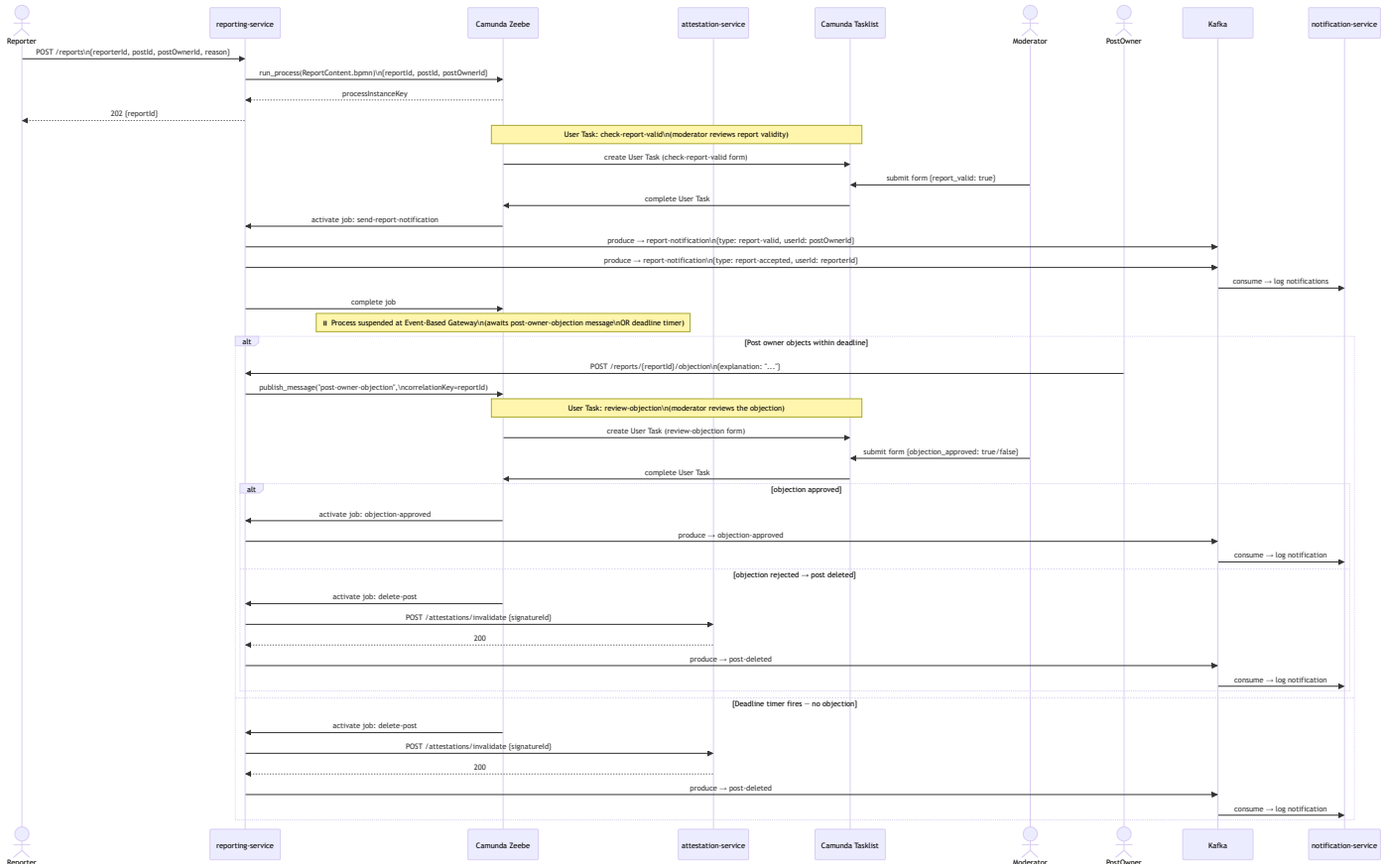


Technically, the `verification-service` is the most orchestration-heavy service in the system. It combines API handling, Zeebe process starts, worker execution, HTTP calls to other services, and Kafka publishing. Its central process variable `verificationId` is a client-visible identifier and the correlation key for Zeebe messages, which keeps the API contract and the workflow state aligned. The process also demonstrates why the platform uses Camunda 8 rather than plain event choreography: peer review is long-running, externally driven, and failure-sensitive, so the system benefits from an explicit process state.

1.5 ReportContent Process

`ReportContent.bpmn` (`Process_0rsygf3`) covers the moderation and governance dimension of the platform (see Appendix D for the visual BPMN representation). A reporter can flag a post as problematic, and the process begins with moderator validation, informs the affected post owner, opens a formal objection window, and handles the outcome depending on whether an objection is filed and how it is reviewed. This process gives the platform a procedural fairness which is explainable and contestable.

The flow combines human judgement, time-based waiting, asynchronous callbacks, and compensating action on related infrastructure state. If the post is ultimately invalidated, the attestation signature is updated through the `attestation-service` .



In technical terms, the `reporting-service` demonstrates the strongest human-in-the-loop characteristics of the platform. It starts the process, participates as Zeebe worker, correlates external objection messages back into the workflow, calls the `attestation-service` for signature invalidation, and publishes all externally visible outcomes to Kafka.

1.6 Technical Architecture and Runtime Setup

The entire stack is orchestrated using docker-compose. From a communication perspective, the project combines three integration styles. First, HTTP/REST is used for user-facing APIs and for selected synchronous service-to-service checks, such as `verification-service` calling `user-service` or either domain service calling `attestation-service`. Second, Zeebe gRPC is used for process starts, job activation, job completion, and message publication. Third, Kafka is used for notification-style events that should remain decoupled from the success of the core flow.

The five services are implemented in Python 3.12 with Flask. The three orchestrated services connect to Zeebe through `pyzeebe`, an asyncio-native Python library that supports process execution and job-worker behaviour. Each of them runs a Zeebe worker in a dedicated background loop and exposes an HTTP API for starting or interacting with process instances. The `attestation-service` is deliberately simpler: it is a REST-only service that stores and invalidates signatures. The `notification-service` has the opposite profile: it exposes no HTTP interface at all and exists purely as a Kafka consumer that simulates user-facing notifications through log output.

At the current project scale, all service-local state is stored in memory. A service restart clears local read-side state, while the authoritative workflow state remains in Zeebe. For the scope of this project we found this to be acceptable.

2. Concepts from the Lecture and Exercises

2.1 Event-Driven Architecture and Streaming

We use Kafka only for notification-style side effects, where loose coupling and replayability matter more than end-to-end control. `user-service` , `verification-service` , and `reporting-service` publish events (`user-registered` , `verification-notification` , `report-notification` , `post-deleted` , `objection-approved`); `notification-service` consumes them independently. ADR-0005 covers why we chose Kafka over a traditional MQ: retained logs and independent consumer groups fit better than broker-managed push.

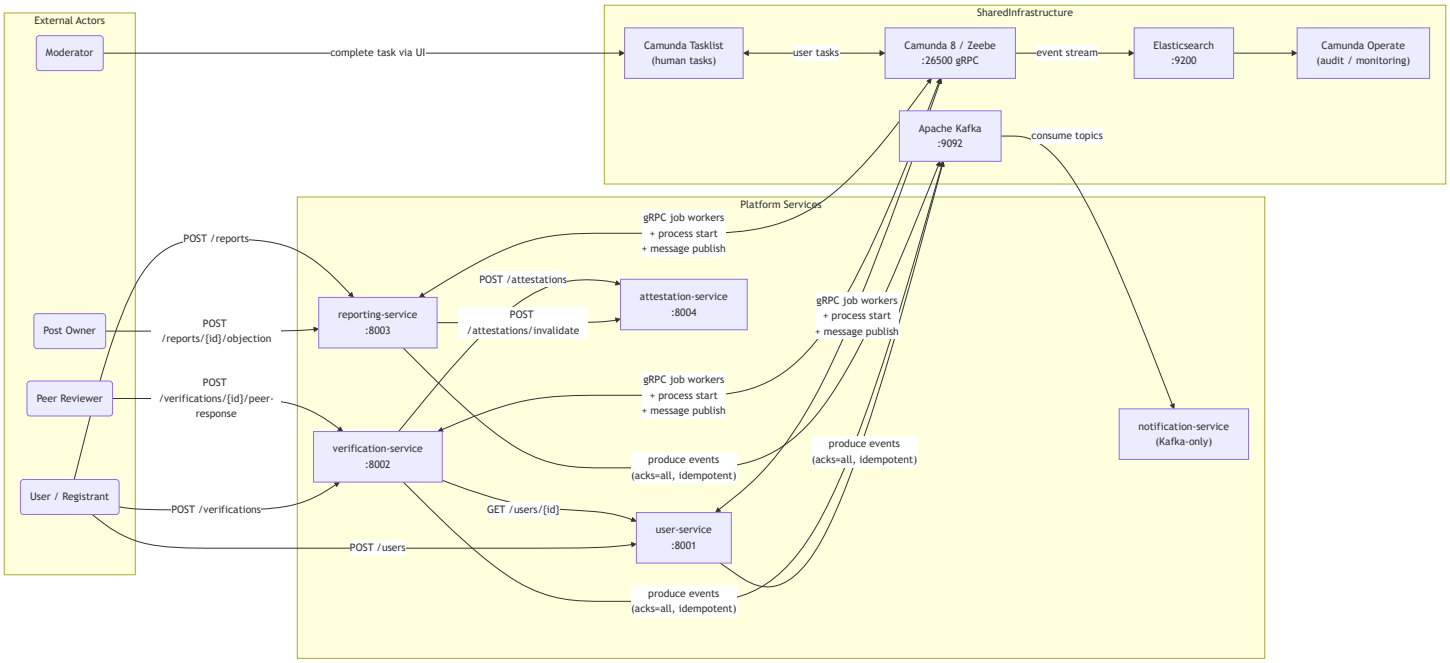
`notification-service` uses a named consumer group with manual offset commits, keeping recovery under application control. Producers use `acks=all` and idempotence, which is a small latency cost for reliability under retries.

We deliberately didn;t create the core verification and moderation logic as topic subscriptions — that would scatter control flow and error handling across services.

2.2 Process-Oriented Architecture and Orchestration

User onboarding, content verification, and reporting all involve explicit waits, branching, deadlines, and outcomes with legal or reputational weight. These need a process owner. `RegisterUser.bpmn` , `VerifyContent.bpmn` , and `ReportContent.bpmn` run as first-class BPMN processes on Camunda 8 (Zeebe), not as implicit event-handler chains.

ADR-0001 captures the split: orchestration for stateful, conditional, auditable flows; choreography for notifications and non-blocking side effects. Domain logic stays in the services, whiule the engine only coordinates the steps.



The BPMN models use service tasks with explicit worker types, user tasks with forms, event-based and exclusive gateways, timer events, and message catch events. ADR-0002 explains Camunda 8 over 7: our codebase is Python, and `pyzeebe` gave us a clean native integration.

2.3 Stateful Resilience and Human Intervention

`VerifyContent.bpmn` and `ReportContent.bpmn` are long-running and depend on state surviving crashes, redeploys, and delayed external input. Instead of writing checkpointing logic, we rely on Zeebe's event-sourced execution: the engine persists process state as an append-only log and reconstructs from snapshots (ADR-0003).

The retry counter `send_verify_retry` is a process variable, not an in-memory worker variable, so timeouts and retry budgets stay correct even if a worker dies mid-wait. This is the difference between worker-local retries and workflow-managed retries.

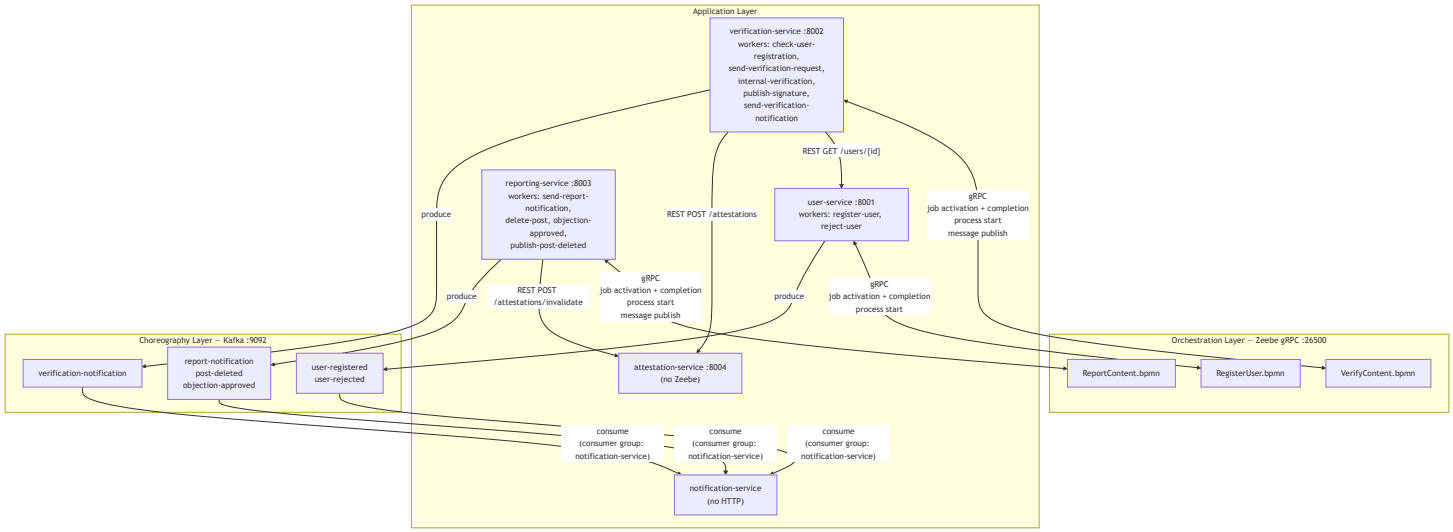
Some failures shouldn't trigger blind automation. The reporting flow pauses at `check-report-valid` and `review-objection`, and registration uses `verify-user` as a human gate.

2.4 Sagas, Eventual Consistency, and Async Correlation

Each service owns its state; cross-service coordination happens through orchestration and messaging (ADR-0004). `VerifyContent` is an orchestrated saga — user registration is checked synchronously via `user-service`, attestation is delegated to `attestation-service`, and failure paths are handled explicitly.

Local views can diverge temporarily: an attestation can be stored while a service's cache hasn't caught up, or a restarted service can lose its read cache while the process state in Zeebe is fine. The cost of eventual consistency is explicit workflow state management; the benefit is responsiveness and service autonomy.

Async message correlation (ADR-0007): instead of polling for peer verdicts or objections, workflows suspend at message catch events and resume when the relevant REST endpoint publishes a correlated Zeebe message. `verificationId` and `reportId` are both API identifiers and workflow correlation keys.



REST when a task needs an immediate answer to advance; async messaging when the process can wait or a side effect shouldn't block completion.

2.6 CQRS and Auditability

The platform makes decisions that need to be explainable later: users get rejected, content gets certified, reports lead to deletion, late objections get ignored. Camunda Operate and Elasticsearch act as a CQRS-style read side: Zeebe emits the process event stream, Operate projects it into a queryable model, Tasklist handles human tasks. The cost is operational weight; the benefit is no bespoke audit DB and much stronger diagnostics than logs alone.

3. Results and Insights

3.1 Kafka

`acks=all` in a single-broker dev setup is effectively `acks=1` but keeps the code production-ready. With `enable.idempotence=True`, no notification is lost or duplicated under retries.

Manual offset commits in `notification-service` mattered during testing. With auto-commit, a crash between receiving and processing a message silently drops it — the offset has already moved. Committing after the handler gives at-least-once processing.

Kafka UI at `localhost:8079` was used constantly for inspecting payloads, verifying topics, and checking consumer lag — much harder with a traditional queue.

3.2 Camunda and Zeebe

One of the more counterintuitive lessons from working with Zeebe was the behaviour of in-memory service state after a restart. The `_verifications` and `_reports` dictionaries in the application services are populated when a process is started via the REST API. If a service container restarts, those dictionaries are empty and any subsequent `GET` request for a running verification or report returns 404, even though the process instance is still alive in Zeebe. This is an accepted limitation at the current project scale and is documented in ADR-0003, but it was initially surprising and underscores the importance of treating Zeebe as the authoritative state store rather than application-tier memory.

Zeebe's job activation timeout gave us implicit crash recovery. When a container restarted mid-job, Zeebe waited out the timeout and re-offered the job.

Message TTL for late peer verdicts: if a verdict arrives after the verification has timed out, the Zeebe message finds no active subscription and is silently dropped. That's "correct" behaviour, and the REST endpoint returns no error. Giving late peers explicit feedback would need an app-layer state check before publishing.

3.3 Impact of Service Unavailability

If `notification-service` is down, all six orchestrated flows continue. Kafka retains events on disk; on restart, the service reads from its last committed offset and catches up.

If a ZeebeWorker service crashes mid-job, Zeebe's activation timeout returns the job to the queue and a restarted worker picks it up. Invisible to the process instance.

If Zeebe itself goes down, all progress is halted — this is an acknowledged downside of a single-broker setup. A production setup would need a multi-broker, multi-partition Zeebe cluster.

4. Reflections and Lessons Learned

The biggest takeaway was that BPMN makes business logic explicit in a way code doesn't. Every timeout, retry boundary, conditional branch, and human decision is readable directly in the model. When a process misbehaved, opening it in Operate and reading its variable history was easier than grepping logs.

`pyzeebe` made Zeebe genuinely usable from Python — a worker is an `async def` with a `@worker.task(task_type="...")` decorator; gRPC, event loop, and reconnection are all abstracted. The python services themselves were trivial to implement, and were roughly 100

lines each.

In-memory state was fine at course scale but wouldn't fly in production. Each service would need persistent storage so `GET` responses stay consistent with Zeebe. The databases already thhave a place architecturally — they would just need to be wired in.

At-least-once delivery requires every worker to be idempotent. For each handler we had to ask whether running it twice leaves the system consistent. In practice that meant cheap side-effect-free checks before mutations, and keeping `attestation-service` calls and Kafka produces safe to repeat.

5. GitHub Release

Assignment-1 release (project / E2-E5): https://github.com/chechmek/EDPO_FS26_Project/releases/tag/assignment-1

Further information provided in the repository README.md.

Assignment-1 release (Kafka-Tests / E1): https://github.com/MrStarco/EDPO_FS26_E1_Kafka_Tests/releases/tag/assignment-1

Further information provided in the repository README.md.

Appendix

The following documents are attached to this submission PDF as appendices. They are maintained as separate files in the repository and are included here without modification.

Appendix A — Submission files:

Submission — Exercise 1: Kafka Getting Started V2
Combined test results from all three team branches covering producer experiments (batch size and latency, acknowledgment configuration, partition scaling, and parallel load), consumer experiments (processing delay and offset misconfiguration), and broker fault-tolerance with leader election.

Changes in this version: Based on the feedback received, the report was restructured to follow the topic order given in the exercise description. Additional diagrams were added to illustrate key results.

File: submission-exercise-1.md

Submission — Exercise 2: Event Notification Pattern
Report on the Event Notification EDA pattern implementation using a social media simulation. Covers the decoupled Kafka-based flow between three microservices (post-service, interaction-service, notification-service), the EDA design rationale, architecture, and verification instructions.
File: submission-exercise-2.md

Submission — Exercise 3: BPMN Orchestration and Process Design
Report on the pivot to a content verification platform and the design of three orchestrated BPMN processes (VerifyContent, RegisterUser, ReportContent) executed by Python/Flask microservices via Camunda Zeebe, including the standalone Attestation Service.
File: submission-exercise-3.md

Submission — Exercise 4: Orchestration vs. Choreography and ADR
Report on the coordination pattern decision (hybrid orchestration with Zeebe and choreography via Kafka) and the ADR justifying Camunda 8 over Camunda 7 for Python-based microservices.
File: submission-exercise-4.md

Submission — Exercise 5: Stateful Resilience Patterns
Detailed report on the implementation of Stateful Retry and Human Intervention in the BPMN processes, including process-level design rationale and lecture references.
File: submission-exercise-5.md

Appendix B — Team Responsibilities
Full exercise-by-exercise responsibility table with links to the repository and commit history.
File: submission-responsibilities.md

Appendix C — Architecture Diagrams
Complete set of six conceptual diagrams: System Context, DDD Context Map, Service Architecture, and three end-to-end sequence diagrams (RegisterUser, VerifyContent, ReportContent).
File: submission-diagrams.md

Appendix D — BPMN Models and Forms
The complete executable process models and their associated Camunda forms used in the project. These files document the workflow logic directly at the modelling level and complement the textual and diagrammatic explanations in the main submission.

File: submission-bpmn-files.md

Appendix E — Architecture Decision Records
Full text of all eight ADRs covering coordination pattern, workflow engine selection, stateful resilience, distributed data consistency, message broker selection, process model boundaries, asynchronous message correlation, and auditing.

ADR	Title	Main Contribution to the Architecture	Key Trade-off
ADR-0001	Coordination Pattern: Hybrid Orchestration and Choreography	Establishes the hybrid model of Zeebe orchestration plus Kafka choreography	Better visibility and control for core flows at the cost of a larger operational surface
ADR-0002	Workflow Engine Selection: Camunda 8 over Camunda 7	Chooses Camunda 8 because Python services can use `pyzeebe` and benefit from event-sourced state	Stronger workflow support and crash resilience at the cost of Java-based infrastructure
ADR-0003	Stateful Resilience and Error Handling	Puts retries and crash recovery into Zeebe and routes irreversible cases to human tasks	Simpler services and clearer recovery paths, but a hard dependency on Zeebe and Tasklist
ADR-0004	Distributed Data Consistency: Saga Pattern over Distributed ACID Transactions	Enforces database-per-service and saga-style coordination instead of distributed transactions	Independent services and scalable coordination, but temporary inconsistency and explicit compensation
ADR-0005	Message Broker Selection: Apache Kafka for Choreography Events	Picks Kafka for retained, replayable, multi-consumer notification streams	More extensibility and durability, but more operational complexity than a simple queue

ADR	Title	Main Contribution to the Architecture	Key Trade-off
ADR-0006	Process Model Boundaries: One BPMN Model per Bounded Context	Prevents a process monolith by aligning one BPMN model with one bounded context	Better ownership and isolation, but cross-context queries require aggregation
ADR-0007	Asynchronous Message Correlation via Zeebe	Uses native Zeebe message correlation for peer verdicts and objections	No polling and no routing tables, but late messages may be discarded after TTL

Submission - E1 Kafka - Test Results

- Course: Event-driven and Process-oriented Architectures (EDPO), FS2026, University of St.Gallen
- Assignment: Exercise 1 - Kafka Getting Started
- Group 4
  - Evan Martino
  - Marco Birchler
  - Roman Babukh

Quick Access Links

- Repository: https://github.com/MrStarco/EDPO_FS26_E1_Kafka_Tests
- Main branch: https://github.com/MrStarco/EDPO_FS26_E1_Kafka_Tests/tree/main
- Marco branch: https://github.com/MrStarco/EDPO_FS26_E1_Kafka_Tests/tree/marco
- Roman branch: https://github.com/MrStarco/EDPO_FS26_E1_Kafka_Tests/tree/roman
- Evan branch: https://github.com/MrStarco/EDPO_FS26_E1_Kafka_Tests/tree/evan

Scope and Method

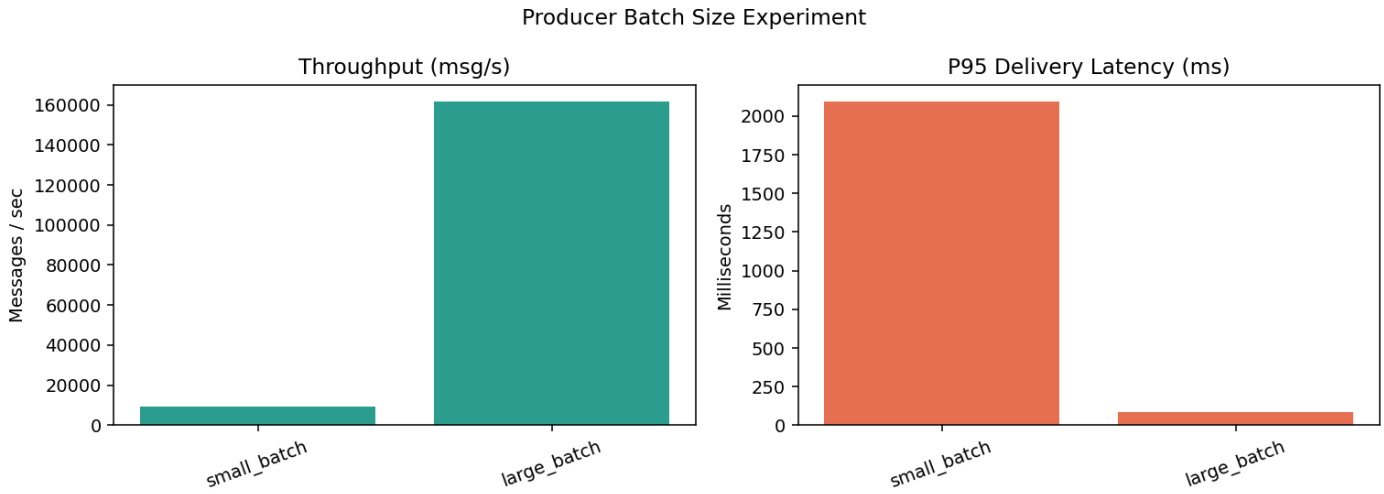
This document is the combined, high-level report. It consolidates results from:

- `marco` branch
- `roman` branch
- `evan` branch

1) Producer Experiments

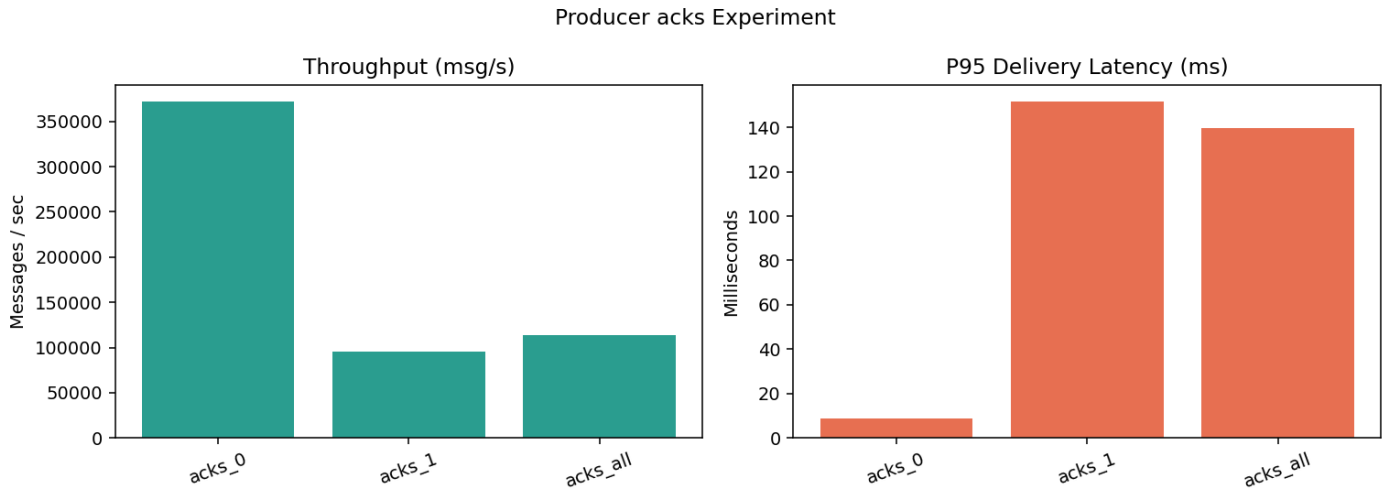
1.1 Batch Size and Processing Latency

- Tested: two producer batching strategies over `20,000` messages, one sending small batches immediately and one collecting much larger batches before sending.
- Result: in the 3-broker setup, the larger-batch strategy performed far better, reaching `161763.76 msg/s` versus `9195.66 msg/s` and lowering p95 latency to `85.241 ms` versus `2095.639 ms`. Neither run produced delivery errors.



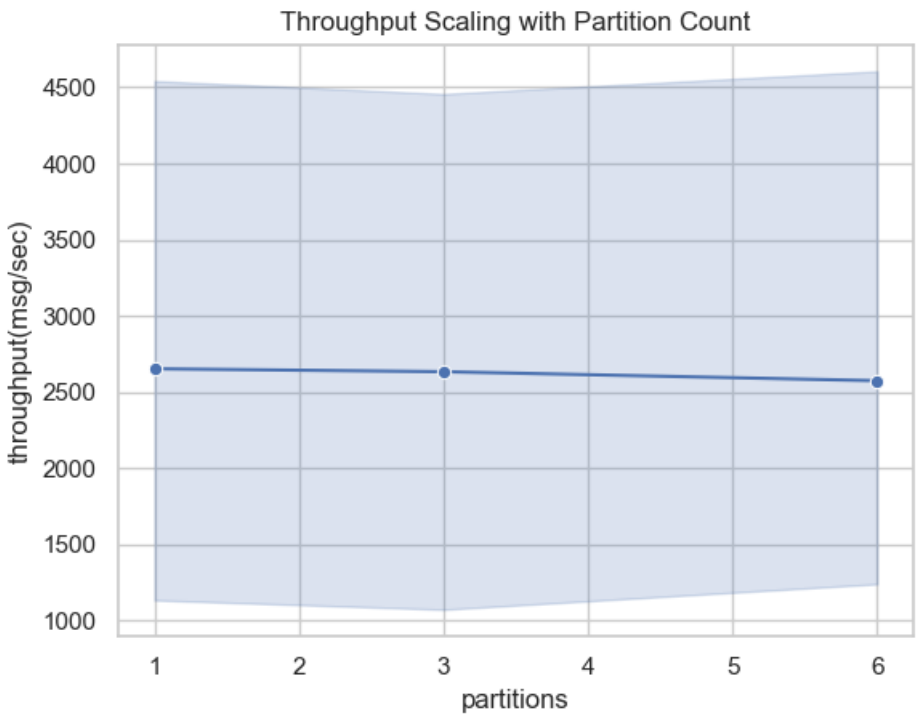
1.2 Acknowledgment Configuration

- Tested: three delivery confirmation levels, from no broker confirmation to full replica confirmation.
- Result: the least durable setting was the fastest in both result sets. In the measured benchmark it reached `371761.9 msg/s` with p95 latency `8.815 ms`, while the two stronger confirmation settings reached `95213.9 msg/s` and `113533.63 msg/s` with p95 latencies of `151.516 ms` and `139.867 ms`. All three runs delivered all `20,000` messages without errors.



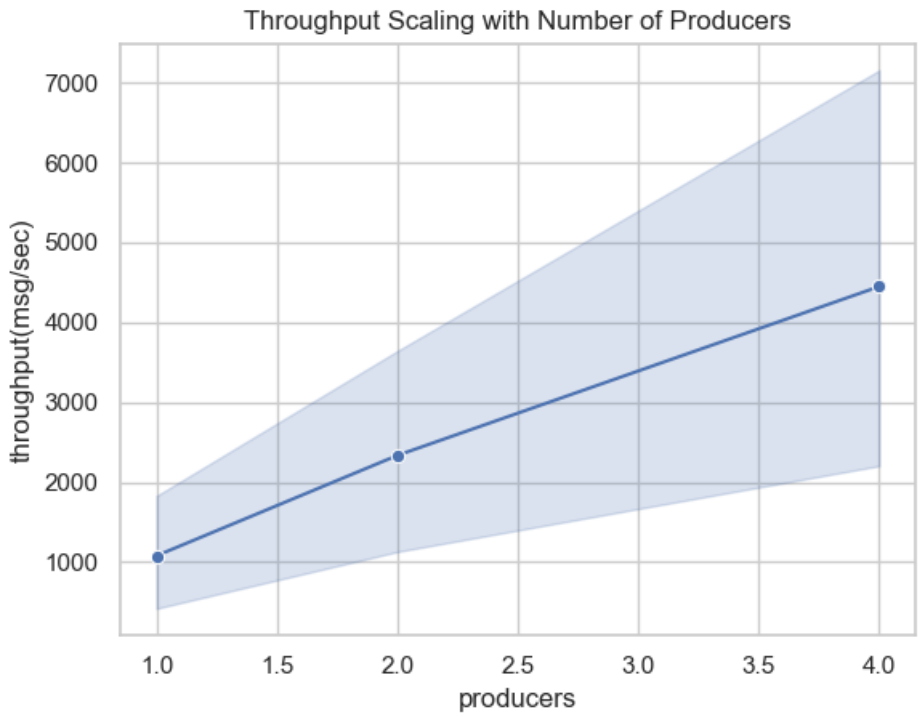
1.3 Brokers and Partitions

- Tested: throughput impact of increasing the number of topic partitions in the single-broker setup.
- Result: adding more partitions did not materially improve throughput. In this setup, all partitions still ran on the same broker and the same laptop hardware, so the bottleneck remained unchanged.



1.4 Load Testing

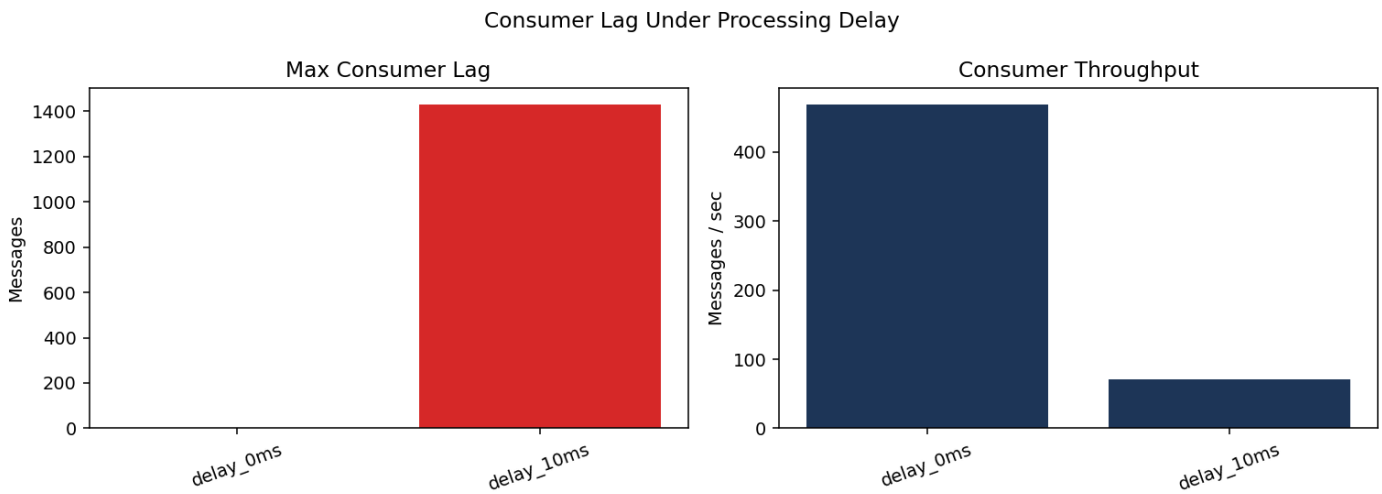
- Tested: throughput scaling as the number of producers increased in parallel.
- Result: throughput increased as more producers ran in parallel. The improvement was smaller when stronger delivery guarantees were enabled, which shows that parallelism helps, but broker confirmation still limits peak throughput.



2) Consumer Experiments

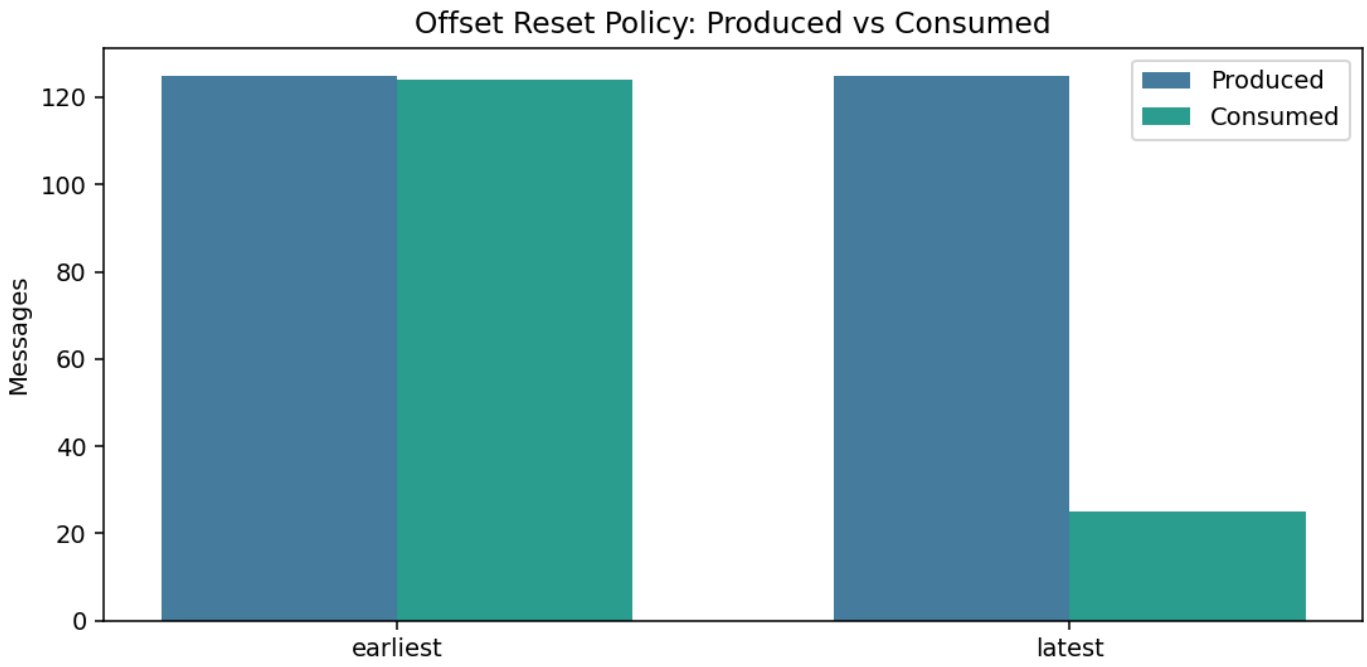
2.1 Consumer Lag Under Processing Delay

- Tested: consumer lag and throughput for `1,500` messages, once with no artificial delay and once with a `10 ms` delay per message.
- Result: the added delay increased maximum lag from `0` to `1431` messages and reduced throughput from `469.32` to `70.83 msg/s`. Both runs still finished all `1,500` messages, but the slower run took `21.179 s` instead of `3.196 s`.



2.2 Offset Misconfiguration

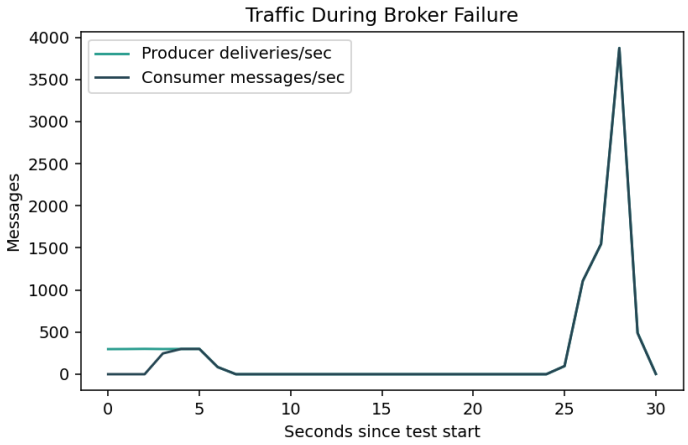
- Tested: a new consumer group joining after `100` messages were already in the topic, once starting from the beginning and once starting only from newly arriving messages. After that, `25` additional messages were produced.
- Result: the consumer that started from the beginning read `99` of the `100` existing messages and all `25` new ones. The consumer that started only from new data read none of the existing messages and all `25` new ones, effectively skipping `100` historical records. This demonstrates the practical data-loss risk of choosing the wrong starting position for a new group.

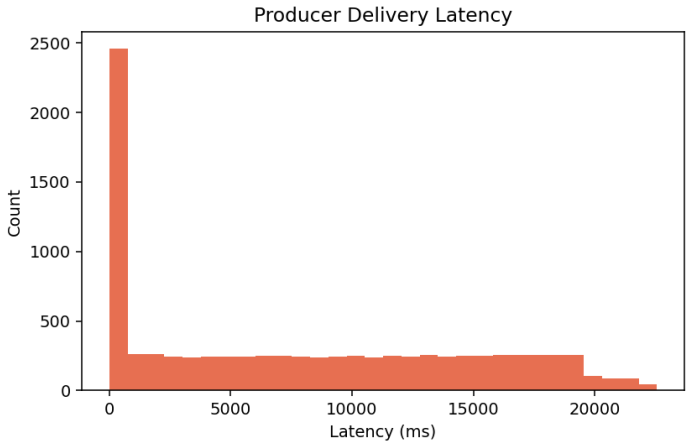


3) Fault Tolerance and Reliability

3.1 Broker Failure and Leader Election

- Tested: broker failover during a `30 s` run on a topic with `6` partitions and replication factor `3`; broker `3` was killed after a `6 s` warmup.
- Result: leader failover affected `2` partitions and took `19.714 s`. The producer still delivered `9001/9001` messages with `0` errors, while the consumer processed `8049` messages and experienced a maximum gap of `19.682 s`. Producer p95 latency spiked to `19176.295 ms` during failover.





Submission - Exercise 2

- Course: Event-driven and Process-oriented Architectures (EDPO), FS2026, University of St.Gallen
- Group 4
  - Evan Martino
  - Marco Birchler
  - Roman Babukh

Repository

- GitHub repository: https://github.com/chechmek/EDPO_FS26_Project

Team contribution

- Roman Babukh focused primarily on the implementation of the project this week.
- Marco Birchler worked on the documentation and submission.
- Evan Martino supported the other team members in both tasks.

Project Abstract

The project demonstrates an event-driven social media simulation using the **Event Notification** EDA pattern, Python 3.12, Kafka, and PostgreSQL with three services:

- `post-service` : stores posts and emits `PostCreated`
- `interaction-service` : stores likes/comments/follows and emits interaction events
- `notification-service` : consumes interaction/post events and builds notifications in its own datastore

EDA Pattern: Event Notification

Definition:

In the Event Notification pattern, a service emits events to inform other services that something has happened in its domain. The emitting service does not know which services (if any) will consume the event, and does not depend on them. Consumers subscribe independently and react to the events they care about.

Key characteristics:

- The producer has no knowledge of consumers
- Events carry only enough information to notify — consumers may need to query back for full state if required
- Communication is asynchronous and one-directional

Why we chose this pattern:

The interaction domain (likes, comments, follows) naturally produces side effects that other parts of the system care about — in particular, notifying post owners. Using Event Notification means `interaction-service` is not coupled to `notification-service` : it simply records facts about user interactions and broadcasts them. This keeps each service focused on its own bounded context and allows the notification logic to evolve independently.

Mapping to our implementation:

- `interaction-service` is the **producer**: it emits `PostLiked`, `PostUnliked`, `CommentAdded`, `UserFollowed` events to Kafka after persisting domain state.
- `notification-service` is the **consumer**: it listens to `social.interaction.events` and `social.post.events` and decides independently which events warrant a user notification.
- There is **no direct synchronous call** from `interaction-service` to `notification-service`. Dependency flows only from consumer to broker, never between services directly.

Architecture

Event Flow

1. Client calls `post-service` or `interaction-service` HTTP APIs.
2. `post-service` persists the post and publishes `PostCreated` to `social.post.events`.
3. `interaction-service` persists the interaction and publishes the corresponding event to `social.interaction.events`.
4. `notification-service` consumes from both topics. It maintains a local `post_owners` table (populated from `PostCreated`) so it can resolve the correct recipient for interaction notifications.
5. For relevant events (`PostLiked`, `CommentAdded`), `notification-service` writes a notification record and exposes it through its own HTTP API.

```
Client
  │
  ├─► POST /posts          → post-service       → Kafka: social.post.events
  │                                                        │
  └─► POST /posts/{id}/like → interaction-service → Kafka: social.interaction.events
                                                           │
                                                  notification-service
                                                  (consumes both topics)
                                                           │
                                                  GET /users/{id}/notifications
```

What Was Implemented for Exercise 2

- Implemented a Kafka-based, decoupled Event Notification flow between interaction and notification domains.
- Implemented publisher logic in `interaction-service` for interaction events (`PostLiked`, `PostUnliked`, `CommentAdded`, `UserFollowed`).
- Implemented subscriber logic in `notification-service` consuming interaction and post events.
- Implemented notification creation for relevant events (`PostLiked`, `CommentAdded`) and non-notifying handling for others.
- Implemented independent data ownership per service (no cross-service DB writes).

How to Verify the Exercise Deliverable

1. Start the system:
   - `make venv`
   - `make install`
   - `make up`
2. Generate activity:
   - `make demo`

- or run `scripts/generate_load.py`

3. Observe event flow:
   - Open Kafka UI at `http://localhost:8080`
   - Inspect `social.post.events` and `social.interaction.events`
4. Verify notification behavior:
   - Query `notification-service` endpoint `GET /users/{user_id}/notifications`
   - Confirm notifications are created for `PostLiked` and `CommentAdded`

Scope and Limitations

- Current notification generation is intentionally limited to owner-facing `PostLiked` and `CommentAdded`.
- `PostUnliked` and `UserFollowed` are consumed but do not create notification records.
- This submission focuses on demonstrating Event Notification and service decoupling for Exercise 2.

Technical Reference

Detailed technical repository documentation (event schemas, data models, API details, setup/runbook) is provided in:

- `README.md`

Submission - Exercise 3

- Course: Event-driven and Process-oriented Architectures (EDPO), FS2026, University of St.Gallen
- Group 4
  - Evan Martino
  - Marco Birchler
  - Roman Babukh

Repository

- GitHub repository: https://github.com/chechmek/EDPO_FS26_Project

Team Contribution

- Evan Martino refined the core concepts and BPMN processes, and made the documentation.
- Marco Birchler created the orignal BPMN processes.

Project Pivot

For Exercise 3, we pivoted from the social media simulation (Exercise 2) to a **content verification platform**. The new system allows users or organizations to submit web content (articles, pages, photos, etc.) for peer-based truth verification. A trusted third-party service coordinates the verification, collects verdicts from independent registered peers, and issues a cryptographic signature for content that passes. The platform also handles user registration and content reporting workflows.

This pivot allows us to focus more on the verification procesess, while purposely leaving the form and justification of the verified content outside of scope (e.g. we don't need to create a news platform, forum, or blog, etc.).

Orchestration

The content verification flow has complex conditional branching (is the user registered? did the peers approve? did the internal check pass?), long waiting periods (peers may respond hours later), and multiple exception paths (timeout, rejection, unregistered user). For these reasons we orchestrated the verification, user registration, and reporting processes.

1. VerifyContent.bpmn

This is the primary orchestrated process. It handles the full lifecycle of a content verification request.

2. RegisterUser.bpmn

Handles onboarding of a new peer-verifier. Registration requires human background review before the user is admitted.

Notable elements:

- **Manual Task** (`Check User Background`): Models a step performed by a human outside the system — in this case a background/trustworthiness check with no system integration. Camunda tracks that the step must happen but does not automate it.

- **User Task** (`Fill out Form`): The applicant submits their details through a Camunda-rendered or custom HTML form. The task carries a Zeebe user task extension so it appears in the task list.
- **Intermediate Message Throw Events**: Both outcome paths end with a message throw, enabling downstream systems (e.g., an email notifier) to react to the registration outcome without being directly coupled to this process.

3. ReportContent.bpmn

Handles a user report against content that has already been published. The process gives the post owner a chance to object before any action is taken.

Notable elements:

- **Event-based Gateway**: The system sends a notification to the post owner and then waits. If the owner raises an `Objection` message (via REST endpoint) before the timer fires, the case is escalated to manual review. If the deadline passes without a response, the post is deleted automatically.
- **Timer Event**: The deadline is a configurable duration (e.g., 72 hours) within Camunda.

Service Architecture

The three BPMN processes are executed by three Python 3.12 / Flask microservices, each owning their own Zeebe job workers:

Service	Port	BPMN process	Zeebe job types
`user-service`	8001	RegisterUser	`register-user` , `send-registration-notification`
`verification-service`	8002	VerifyContent	`check-user-registration` , `send-verification-request` , `internal-verification` , `publish-signature` , `send-verification-notification`
`reporting-service`	8003	ReportContent	`send-report-notification` , `delete-post`
`attestation-service`	8004	(none — scheduler)	—

Each service starts its Zeebe job workers in a background thread alongside the Flask HTTP server. The `verification-service` also exposes `POST /verifications/<id>/peer-response` to receive verdicts from external peers, which it then correlates to the waiting Camunda process instance via `client.publish_message(name, correlation_key, variables)` .

Attestation Service

In addition to the three orchestrated processes, the platform includes a standalone **Attestation Service** (port 8004). This service periodically re-fetches registered content URLs and compares their SHA-256 hash against the baseline captured at verification time. If the content has changed, it marks the attestation as `tampered` and publishes an `AttestationFailed` event to Kafka. This ensures that a published signature remains valid and that content drift is detected automatically.

Technical Reference

- BPMN files: `bpmn files/`
- Service skeletons: `services/`
- Infrastructure: `docker-compose.yml`

Submission - Exercise 4

- Course: Event-driven and Process-oriented Architectures (EDPO), FS2026, University of St.Gallen
- Group 4
  - Evan Martino
  - Marco Birchler
  - Roman Babukh

Repository

- GitHub repository: https://github.com/chechmek/EDPO_FS26_Project

Team Contribution

- Evan Martino refined our core concepts and BPMN processes, and made the documentation.
- Marco Birchler created the orignal BPMN processes.

1. Orchestration vs. Choreography

All three processes (`VerifyContent` , `RegisterUser` , `ReportContent`) are **orchestrated** by Camunda Zeebe. The flows involve complex branching, long waiting periods (peer responses may arrive hours later), and high-stakes outcomes (a published cryptographic signature).

Peer reviewers are **humans**, not autonomous services. The process emails them a review link; they submit their determination via a REST callback that is correlated back to the waiting process instance using `verificationId` as a correlation key. This is **orchestration with asynchronous human-in-the-loop tasks**.

Choreography in our architecture is limited to the **Attestation Service** and **Notification Service** processes. Attestation Service runs on its own schedule and publishes `AttestationFailed` events to Kafka without instructions from an orchestrator. Consumers react independently — although we have not mapped out exactly what would happen. Perhaps it would start a new verification process. Notification service just subrscribes to relevant events.

2. ADR — Camunda 8 (Zeebe) vs. Camunda 7 / Operaton

ADR-001: Workflow Engine Selection

Status: Accepted

Context:

The project requires a workflow engine to execute three BPMN processes with long-running state, message correlation, and timer events. We need to decide between Camunda 7 and Camunda 8.

Decision: Use **Camunda 8 (Zeebe)** with `pyzeebe` job workers.

Justification:

Our services are written in **Python using Flask**, not Java or Spring Boot. This is the single most consequential factor in the decision.

Camunda 7's primary integration model is Java delegates and Spring Boot embedded. External task workers are supported in C7 but are a secondary pattern. Zeebe's job worker model is the primary and idiomatic pattern, and `pyzeebe` (an asyncio-based Python library) provides a clean, well-maintained client. There is no equivalent for C7.

Trade-offs accepted:

- **Infrastructure overhead.** Running Camunda 8 self-managed requires Zeebe, Operate, and Elasticsearch (for process history). This is more infrastructure than an embedded Camunda 7 engine. We accept this because our services are already containerized with Docker. This is a pattern we are very familiar with.
- **Operate dependency for monitoring.** Camunda 8 does not expose a built-in history REST API in the same way C7 does. Operate (requiring Elasticsearch) is needed for process monitoring. For development and demo purposes this is acceptable.
- **Maturity of tooling.** Some Camunda 7 ecosystem features (complex BPMN simulations, decision tables via DMN embedded in processes) have less coverage in Zeebe. We do not use DMN in our current processes, so this is not a blocking concern.

Technical Reference

- BPMN files: `bpmn files/`
- Service skeletons: `services/`
- Infrastructure: `docker-compose.yml`
- Exercise 3 submission (process design detail): `submission-exercise-3.md`

Submission - Exercise 5

- Course: Event-driven and Process-oriented Architectures (EDPO), FS2026, University of St.Gallen
- Group 4
  - Evan Martino
  - Marco Birchler
  - Roman Babukh

Repository

- GitHub repository: https://github.com/chechmek/EDPO_FS26_Project

Team Contribution

- Evan Martino worked on the implementation and deployment of the processes.
- Roman Babukh worked on the implementation of the non-process-related parts of the project.
- Marco Birchler worked on the documentation and submission.

For an overview of responsibilities across all exercises, see `[submission-responsibilities.md](./submission-responsibilities.md)`.

Report on Stateful Resilience Patterns in the Software Project

For our content verification platform we implemented two of the stateful resilience patterns introduced in the lecture and Lab08 (Flowing Retail): **Stateful Retry** and **Human Intervention**. Both patterns are modelled directly in BPMN and executed by Camunda Zeebe. Because Zeebe is an **event-sourced workflow engine**, all changes to process state are recorded in an append-only event log and materialized into RocksDB snapshots (Lecture 6, slides 4-5). The current state of any process instance can be fully reconstructed from this log, meaning process variables survive crashes and redeployments without any additional persistence code on our side (Lecture 6, slide 8). This is the foundational property that makes stateful resilience patterns practical. Zeebe does not implement ACID transactions across services. As discussed in class, this presents two design options: commit to the domain first and then notify Zeebe (at-least-once), or notify Zeebe first and then commit (at-most-once) (Lecture 6, slide 25). We follow the **at-least-once** approach, committing domain state before completing the Zeebe job, accepting the possibility of duplicate processing in exchange for the guarantee that no state update is ever silently lost. Our patterns therefore rely on **state management and eventual consistency** rather than distributed transactions.

2.1 Stateful Retry: `VerifyContent.bpmn`

Implementation: In `VerifyContent.bpmn`, after `Send Verification Request to Peers` completes, an Event-Based Gateway waits for either a peer response message or a configurable timer. On timeout or service error, a Script Task increments `send_verify_retry` and an exclusive gateway routes back to retry or forward to a failure notification once the limit is reached.

Peer verification is the core value proposition of the platform: a cryptographic signature is only meaningful if it was issued after a genuine human review. Peers are humans who may be slow, offline, or simply unresponsive, so transient non-responses are expected and should not immediately cause a verification to fail.

As discussed in class, **stateful retry** differs fundamentally from a simple in-process retry: the retry state is owned by the workflow engine rather than the job worker. This matters here because the retry boundary spans an asynchronous wait that can last minutes. A job-worker retry would be reset by any crash during that window; a Zeebe process variable is not, because the engine reconstructs it from the event log on recovery. The retry count is also fully visible in **Camunda Operate** via the CQRS exporter to Elasticsearch, making the behaviour auditable (Lecture 6, slides 26-27). Capping retries at three prevents indefinite looping; after exhausting retries the requester receives a clear notification rather than a silent failure.

The main tradeoff is added process complexity: the retry loop, counter variable, and boundary error event all need to be modelled and tested explicitly. We accepted this cost because silently dropping a failed verification or failing on the first timeout would undermine trust in the platform.

2.2 Human Intervention: `ReportContent.bpmn`

Implementation: In `ReportContent.bpmn`, after the post owner is notified, an Event-Based Gateway waits for either an `Objection` message or a deadline timer. The objection message uses Zeebe's **message correlation** mechanism: a subscription is registered on a partition determined by a hash of `reportId`, so the arriving message is reliably routed back to the correct waiting process instance (Lecture 6, slide 24). If an objection arrives, a `Review Objection` User Task is created in Camunda Tasklist for a moderator, who approves or rejects it via a rendered form. If the deadline fires with no objection, the post is deleted automatically.

Content moderation is an inherently subjective judgement that carries legal and reputational risk. As covered under **human intervention** in class, there are decisions that should not be fully automated: the consequences are irreversible (deletion) and the inputs are ambiguous (a contested report). By suspending the process at an Event-Based Gateway and giving the post owner a fixed window to raise an objection, we provide due process before anything is deleted. If an objection arrives, a moderator makes the final call rather than the system. This matches the human-in-the-loop pattern from the lecture, where a User Task pauses automated flow until a human provides a decision.

The tradeoff is that the process may stay open for the full deadline period before acting, but this is an intentional and explainable delay rather than a system failure.

References

Technical References

- BPMN files: `[bpmn files/](../bpmn%20files/)`
- Service skeletons: `[services/](../services/)`
- Infrastructure: `[docker-compose.yml](../docker-compose.yml)`
- Exercise 3 submission (process design detail): `[submission-exercise-3.md](./submission-exercise-3.md)`
- Exercise 4 submission (orchestration & ADR): `[submission-exercise-4.md](./submission-exercise-4.md)`

Images of Processes and Forms

- VerifyContent process (stateful retry): `[images/E5/VerifyContent_E5.png](./images/E5/VerifyContent_E5.png)`
- ReportContent process (human intervention): `[images/E5/ReportContent_E5.png](./images/E5/ReportContent_E5.png)`
- RegisterUser process: `[images/E5/RegisterUser_E5.png](./images/E5/RegisterUser_E5.png)`
- Verify user: `[images/E5/VerifyUser_E5.png](./images/E5/VerifyUser_E5.png)`
- Check report validity: `[images/E5/CheckReportValidity_E5.png](./images/E5/CheckReportValidity_E5.png)`
- Review objection: `[images/E5/ReviewObjection_E5.png](./images/E5/ReviewObjection_E5.png)`

Appendix B - Team Responsibilities

- Course: Event-driven and Process-oriented Architectures (EDPO), FS2026, University of St.Gallen
- Group 4
  - Evan Martino
  - Marco Birchler
  - Roman Babukh

Further Information

- GitHub Repository: chechmek/EDPOFS26Project
- Commit History: github.com/chechmek/EDPOFS26Project/commits/main

Responsibilities

Exercise	Task	Responsible	Description
1	Experimenting	Evan	Producer-focused experiment runs and review on Evan
1	Experimenting	Marco	Experiment design and execution on Marco
1	Experimenting	Roman	Experiment framework and outputs on Roman
1	Revision	Roman	Revised parts of Exercise 1 according to feedback
2	Implementation	Roman	Focused primarily on the initial implementation of the project
2	Documentation	Marco	Worked on the documentation and submission
2	Support	Evan	Supported the other team members in both tasks
3	BPMN	Marco	Created the original BPMN processes
3	Improving	Evan	Refined the core concepts and BPMN processes
3	Documentation	Roman	Worked on the documentation and submission
4	Improving	Marco	Refined the core concepts and BPMN processes
4	Improving	Evan	Refined the core concepts and BPMN processes
4	Documentation	Roman	Worked on the documentation and submission
5	Implementation	Evan	Worked on the implementation and deployment of the processes
5	Implementation	Roman	Worked on the implementation of the non-process related parts of the project
5	Documentation	Marco	Worked on the documentation and submission

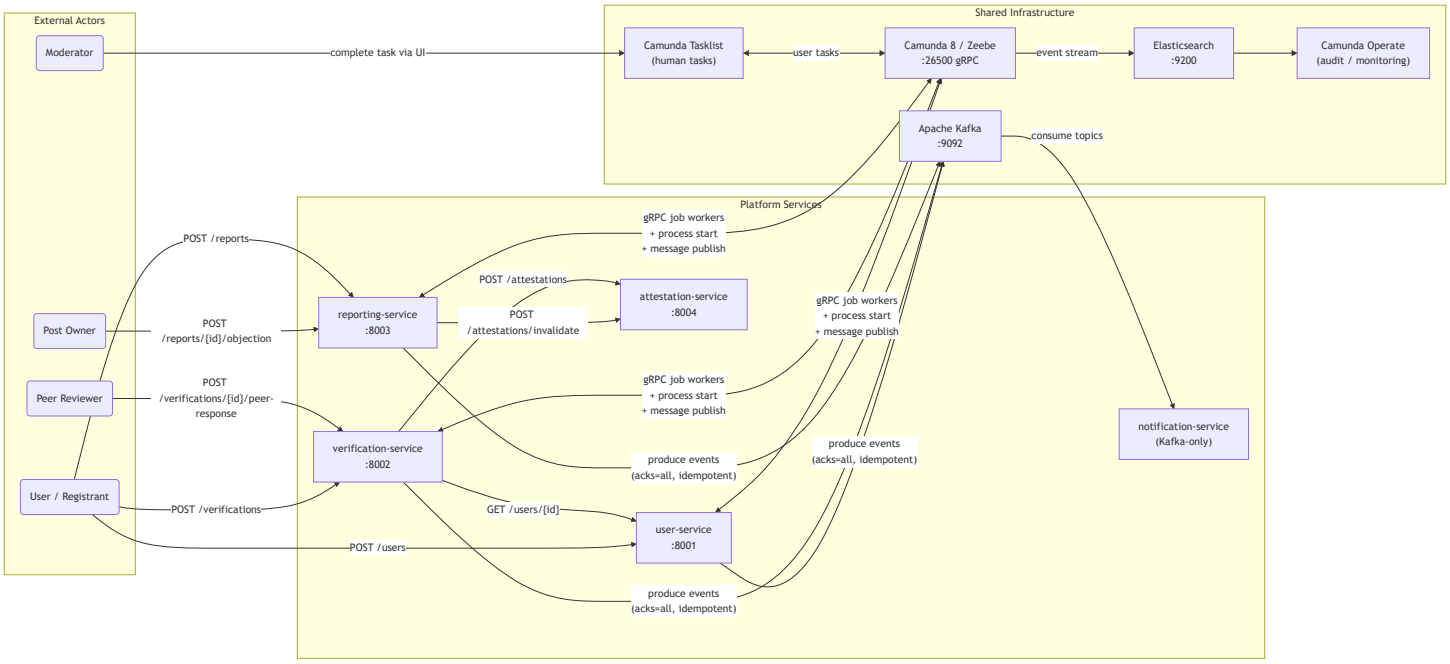
Appendix C - Architecture Diagrams

- Course: Event-driven and Process-oriented Architectures (EDPO), FS2026, University of St.Gallen
- Group 4
  - Evan Martino
  - Marco Birchler
  - Roman Babukh

This document provides a set of conceptual-level diagrams covering the full implementation. The diagrams progress from the broad system view down to specific interaction patterns. Each is followed by a short explanation of what it shows and why it matters for the architecture.

1. System Context

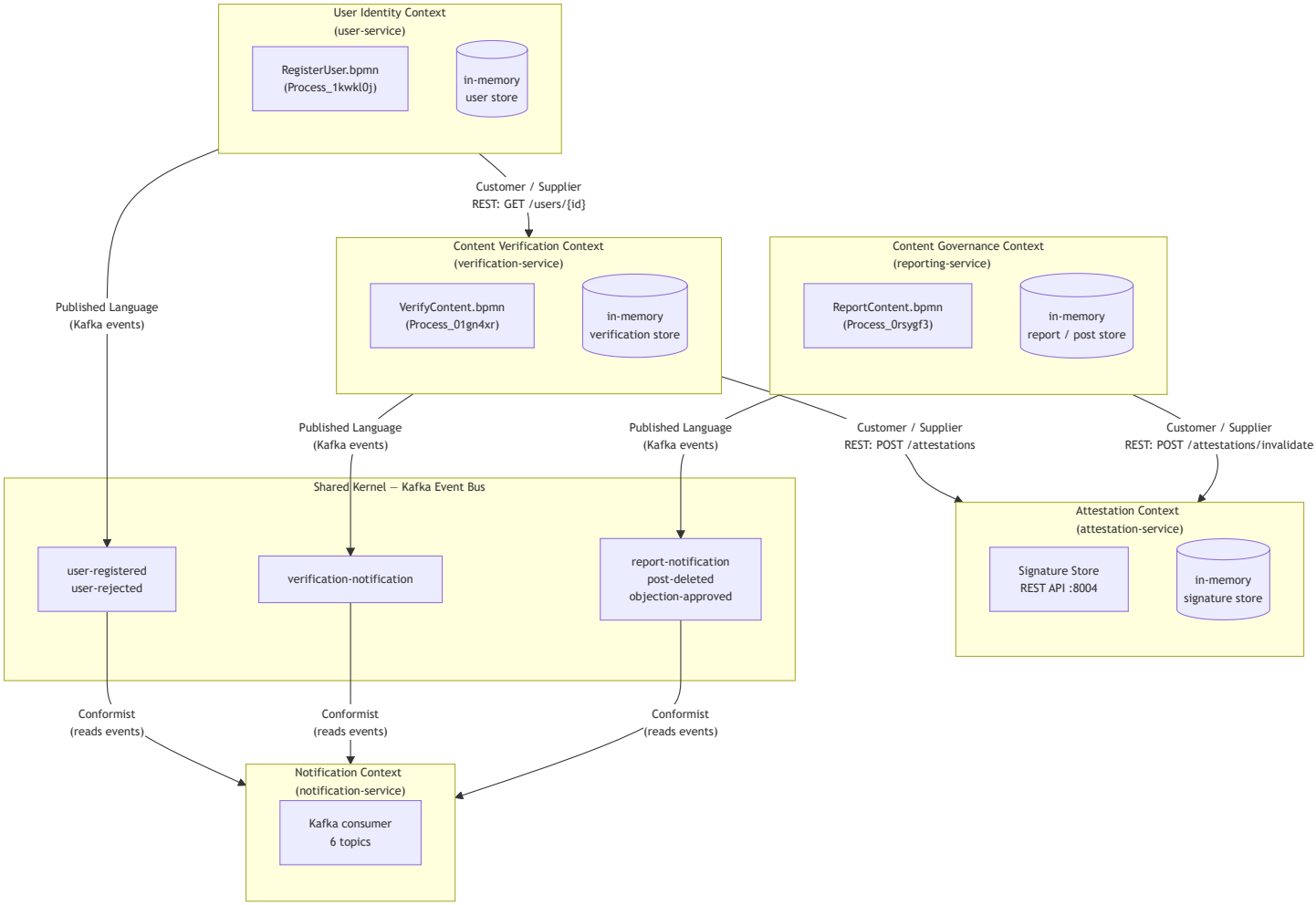
The highest-level view: who interacts with the platform and through which channel. External actors on the left; platform services in the middle; shared infrastructure on the right.



Reading notes: All three orchestrated services (`user-service` , `verification-service` , `reporting-service`) maintain a bidirectional gRPC connection to Zeebe — they start processes, activate jobs, complete jobs, and publish messages over the same connection. The `attestation-service` is the only service with no Zeebe involvement; it is purely a REST-based signature store. The `notification-service` has no HTTP interface at all and only ever reads from Kafka.

2. Context Map (DDD)

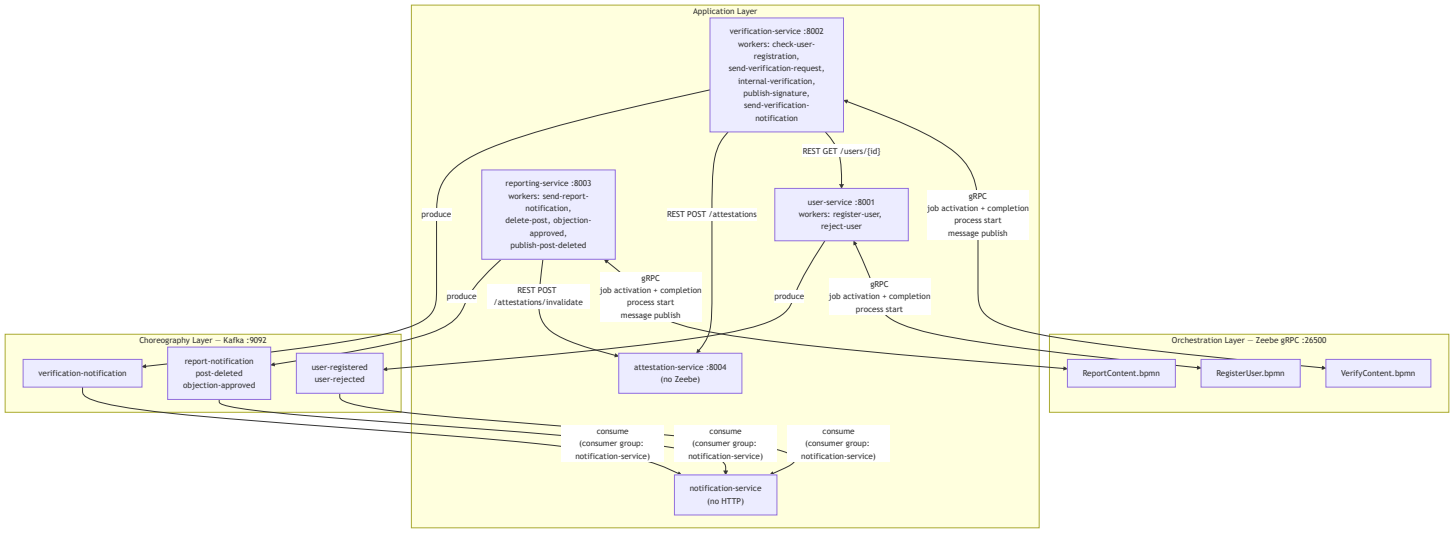
The platform has four bounded contexts. This diagram shows their boundaries and the type of relationship between them, using standard DDD notation.



Reading notes: The `User Identity` context is upstream of `Content Verification` — verifications depend on it to check eligibility, but the user service has no knowledge of verifications. The `Attestation` context is a shared capability used by both `Content Verification` (to store signatures) and `Content Governance` (to invalidate them), but it has no dependency on either. Kafka serves as a shared event bus for loosely coupled notifications following a Published Language pattern; the `notification-service` is a conformist consumer that adapts to whatever events the producing contexts emit.

3. Service Architecture and Communication Channels

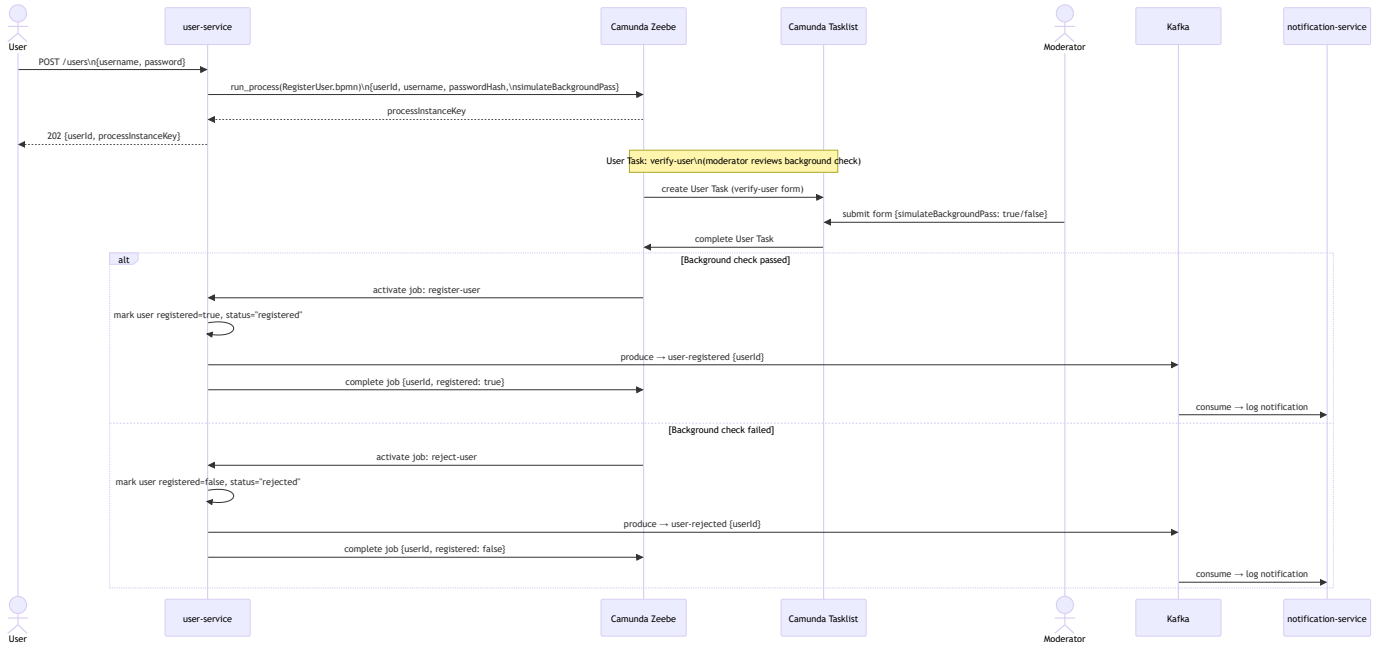
This diagram makes the three different communication channels explicit: Zeebe gRPC (orchestration), synchronous REST (service-to-service calls), and Kafka (choreography).



Reading notes: Each service owns exactly the job types declared in its own BPMN model; there is no cross-service worker. The gRPC connection carries four distinct interaction types: job polling (`ZeebeWorker`), job completion, process instance launch (`ZeebeClient.run_process`), and message publication (`ZeebeClient.publish_message`). REST calls are used only for point-to-point queries where a synchronous answer is needed within a job handler — they are never used across process boundaries for coordination.

4. RegisterUser — Sequence

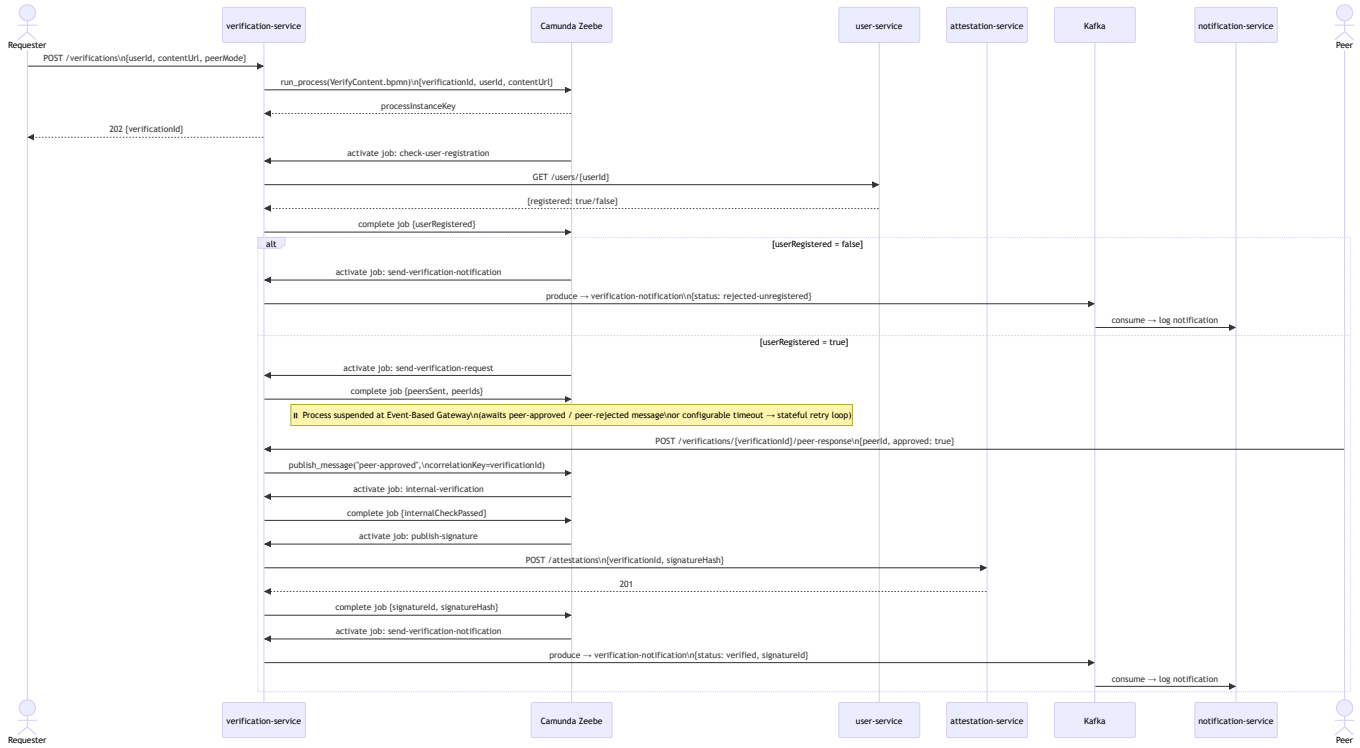
The user onboarding flow. Background verification is modelled as a human task in Tasklist, making the approval decision explicit, auditable, and not automated away.



Reading notes: The `simulateBackgroundPass` flag lets us drive both paths in tests without requiring a real background-check integration. In production this would be replaced by a genuine third-party check result. The `userId` generated at process start is the stable identifier used in all downstream flows — it is the correlation between the identity context and the verification context.

5. VerifyContent — End-to-End Sequence

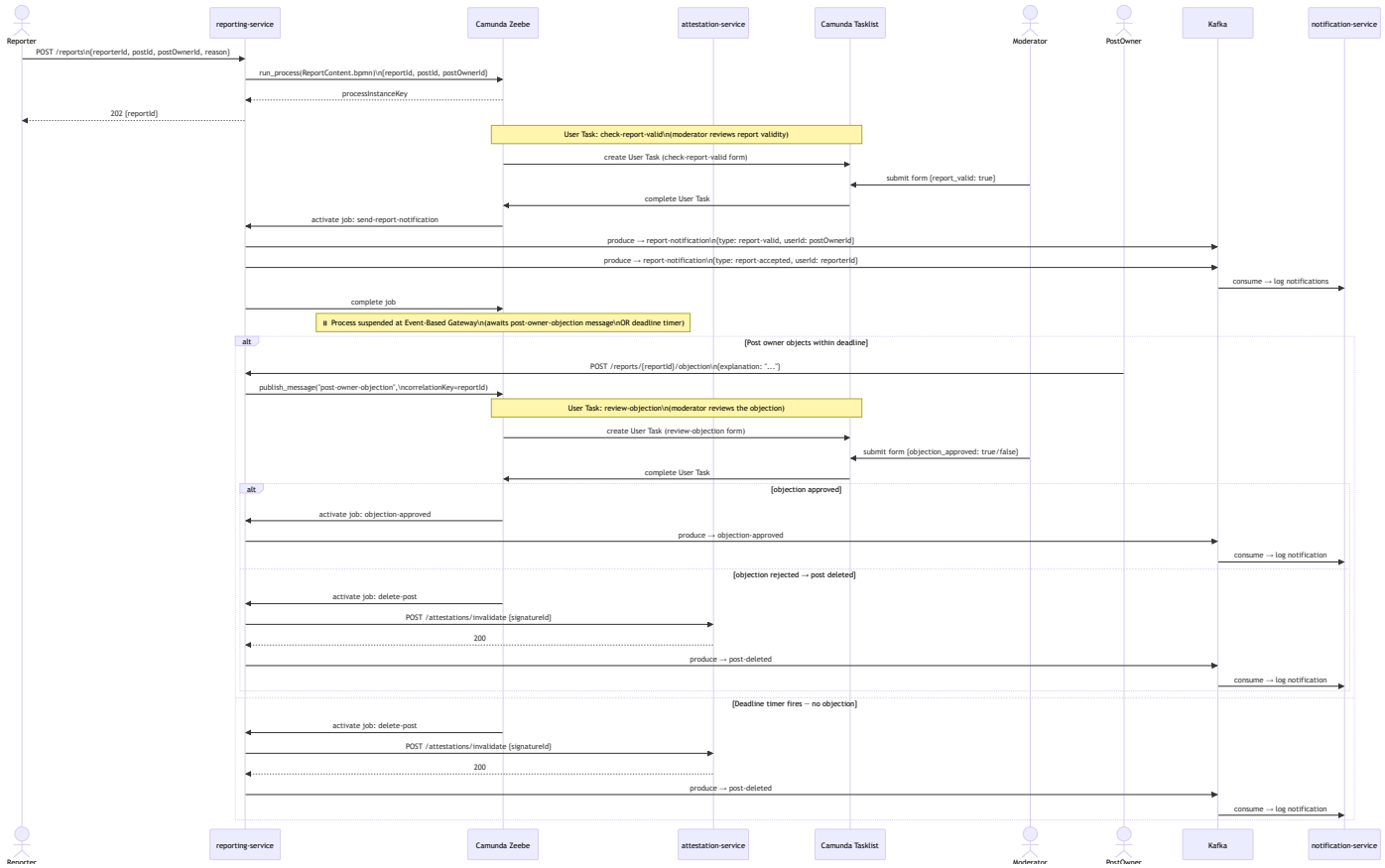
The full lifecycle of a content verification, from API call to Kafka notification. This is the most complex flow and demonstrates orchestration, message correlation, service-to-service REST, and Kafka choreography in a single trace.



Reading notes: The process instance is suspended at the Event-Based Gateway waiting for a peer message or a timeout. During that suspension, zero CPU or memory is consumed by the application — the state lives entirely in Zeebe's event-sourced log. The `verificationId` UUID is both the client-facing identifier (used in the REST URL) and the Zeebe correlation key, which eliminates any secondary routing table.

6. ReportContent — Human Intervention Sequence

The reporting flow demonstrates the human-in-the-loop pattern. A moderator's User Task gates the objection window, and a post owner's REST callback correlates back to the suspended instance.



Reading notes: There are two distinct human tasks in this flow. The first (`check-report-valid`) is always required — no automation replaces the moderator's initial judgement. The second (`review-objection`) is conditional: it only fires if the post owner submits an objection within the deadline. Both User Tasks are visible in Camunda Tasklist with rendered forms, and both completions are recorded in Camunda Operate's audit history.

Summary Table

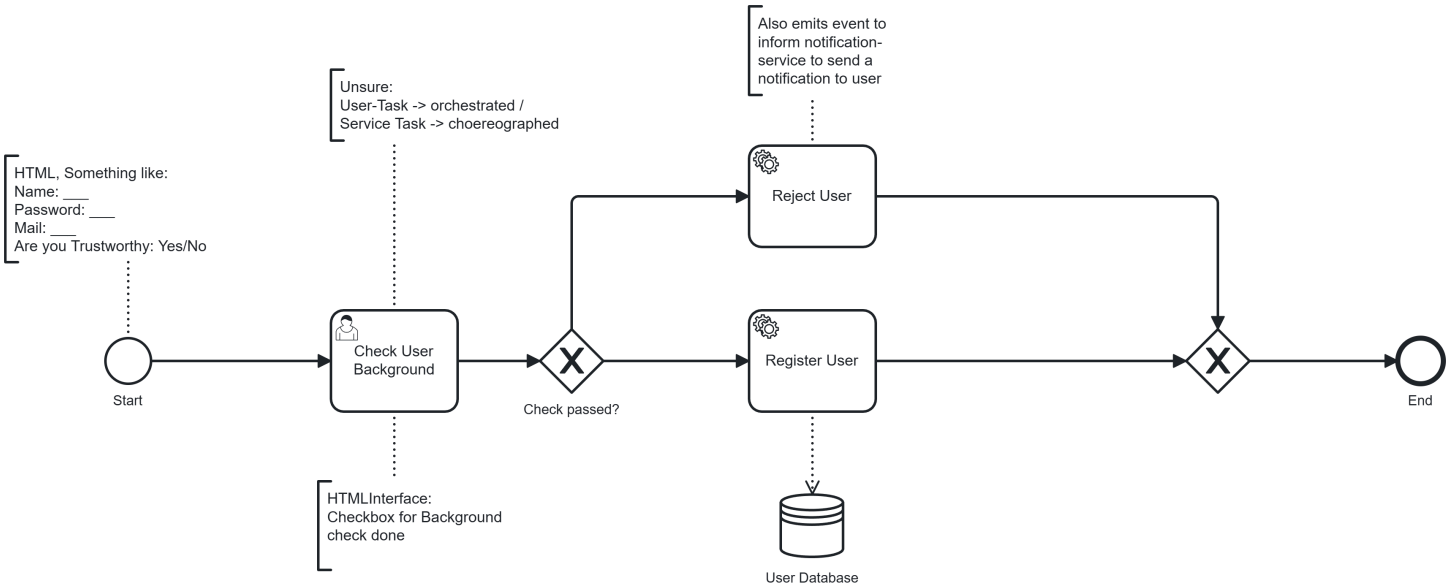
Diagram	Pattern / Concept	ADR Reference
1. System Context	Overall platform boundary and actors	—
2. Context Map	Bounded contexts, upstream/downstream relationships	ADR-0006
3. Service Architecture	Communication channels: gRPC, REST, Kafka	ADR-0001, ADR-0005
4. RegisterUser Sequence	Human task onboarding, Kafka notification	ADR-0003, ADR-0006
5. VerifyContent Sequence	Orchestration, message correlation, Kafka	ADR-0007, ADR-0005
6. ReportContent Sequence	Human intervention, objection correlation, deletion saga	ADR-0003, ADR-0007

Appendix D - BPMN Models and Forms

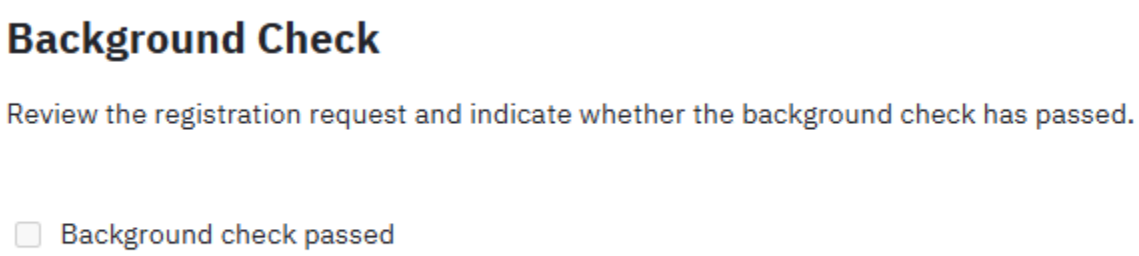
- Course: Event-driven and Process-oriented Architectures (EDPO), FS2026, University of St.Gallen
- Group 4
  - Evan Martino
  - Marco Birchler
  - Roman Babukh

This document provides visual representations of the executable BPMN processes and their associated Camunda forms used in the project.

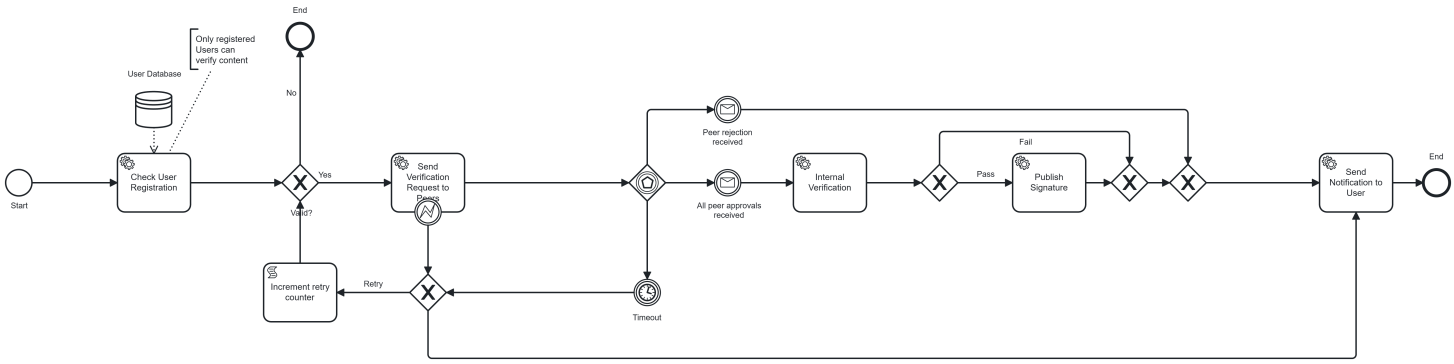
1. RegisterUser Process



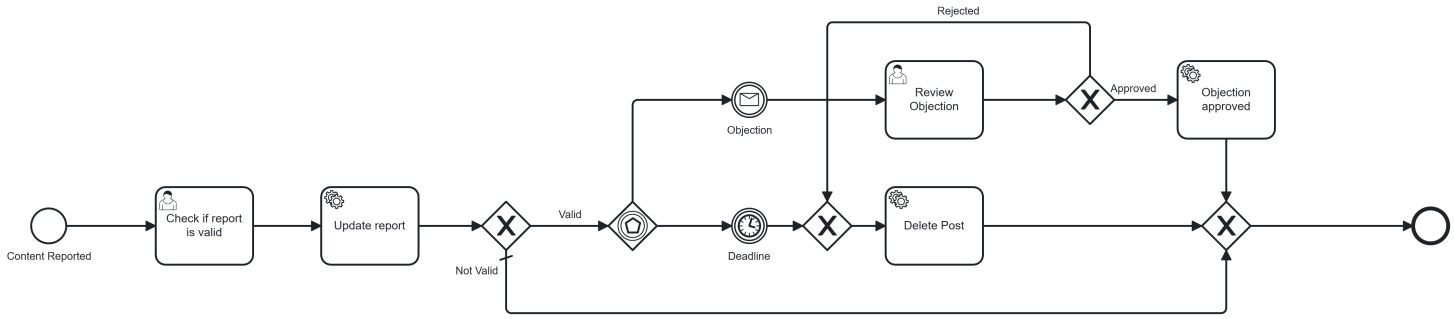
1.1 Verify User Form



2. VerifyContent Process



3. ReportContent Process



3.1 Check Report Validity Form

Check Report Validity

Review the submitted content report and determine whether it describes a genuine policy violation.

- [ ] Report is Valid

3.2 Review Objection Form

Review Objection

The post owner has objected to this report. Review their objection and determine whether it should be rejected (post is deleted) or upheld (post remains).

- [ ] Reject Objection

0001 - Coordination Pattern: Hybrid Orchestration and Choreography

Status

Accepted

Context

Our platform needs to coordinate work across five services: `user-service`, `verification-service`, `reporting-service`, `attestation-service`, and `notification-service`. These involve three distinct business flows: user registration, content verification, and content reporting. These are multi-step, stateful flows where we need to know at any moment exactly where a given request stands.

The notification flow, on the other hand, is a side-effect. When a verification completes or a post gets deleted, any number of downstream consumers need to be informed, but none of them need to coordinate with each other, and no consumer failing should block the primary flow from completing.

Options Considered

- **Option 1**: Pure Orchestration. Pros: Centralized control, easy to monitor. Cons: Tight coupling, single point of failure.
- **Option 2**: Pure Choreography. Pros: Loose coupling, highly scalable. Cons: Hard to monitor, complex error handling.
- **Option 3**: Hybrid Approach (Chosen). Pros: Best of both worlds. Cons: Increased complexity.

Decision

We adopt a hybrid coordination strategy: Camunda 8 (Zeebe) orchestrates all three core business processes `RegisterUser.bpmn`, `VerifyContent.bpmn`, and `ReportContent.bpmn` while Apache Kafka handles the loosely coupled, side-effect notification flows.

The rule for which pattern to apply is straightforward. Any flow with complex conditional branching or asynchronous waits with defined timeouts belongs in a BPMN model under Zeebe's control. All three of our business processes satisfy these criteria. The content verification flow alone involves a user-registration check, dispatching requests to peers, waiting for their verdicts (with a timeout), running an internal check, calling the attestation service, and finally notifying the requester. Each step represented as an explicit BPMN task with its own retry and failure semantics.

Notifications such as `verification-notification`, `report-notification`, `user-registered`, `user-rejected`, `post-deleted`, and `objection-approved` are published to Kafka topics by the relevant service once the orchestrated step completes. The `notification-service` subscribes to all of these topics independently. This service has no HTTP interface at all; it is a pure Kafka consumer. If it restarts or falls behind, the primary workflows are completely unaffected, and no messages are lost because Kafka retains the log.

0002 - Workflow Engine Selection: Camunda 8 over Camunda 7

Status

Accepted

Context

We need a BPMN workflow engine to orchestrate our three core business processes. The two candidates in the Camunda ecosystem are Camunda 7 (the long-established, Spring Boot native platform) and Camunda 8 (the newer, Zeebe-based engine built around an event-sourced, distributed architecture).

Our team writes Python 3.12. None of our services use Java or Spring Boot, and we have no intention of introducing either.

Camunda 7 was designed around a Java process engine. Its REST API can technically be called from any language, but the available Python clients are thin wrappers that require managing raw HTTP, polling, and serialization by hand. There is no officially supported, idiomatically Python job-worker client for Camunda 7. Running job workers against it from Python means either a bespoke polling loop or a third-party library with no active maintenance.

Camunda 8 has `pyzeebe`, an actively maintained, asyncio-native Python library that provides first-class job-worker support, message publication, and process lifecycle management over gRPC. This is not a workaround - it is the intended integration path for Python services.

The second relevant dimension is state persistence. Camunda 7 stores process state in a relational database (typically PostgreSQL or MySQL). All variable mutations, token movements, and wait states require transactional writes to a shared schema. Under high concurrency this becomes a bottleneck and a single point of failure. Camunda 8's engine (Zeebe) uses an event-sourced log persisted to RocksDB, partitioned across brokers. Process state is the accumulated result of an append-only log; the engine never needs to update rows, only append records.

Options Considered

Camunda 7 has a mature ecosystem and strong tooling, but its process engine is Java-native. Its REST API can be called from Python, but there is no officially supported, idiomatic Python job-worker client. Running workers from Python requires either a hand-rolled polling loop or an unmaintained third-party library. Process state is stored in a relational database, which becomes a write bottleneck under high concurrency. Rejected on grounds of interoperability and scalability.

Camunda 8 / Zeebe (chosen) provides `pyzeebe`, an actively maintained asyncio-native Python library with first-class job-worker support over gRPC. Its event-sourced, append-only persistence to RocksDB removes the relational write bottleneck. Process state survives worker crashes by design, with no application-level persistence code required.

Decision

We use Camunda 8 (Zeebe), operated in self-managed mode via `c8run`, as the workflow engine for all three BPMN processes. Our Python services interact with it exclusively through `pyzeebe`, which is used in all three orchestrated services: `user-service`, `verification-service`, and `reporting-service`. Each service runs a `ZeebeWorker` in a dedicated asyncio event loop on a background thread, and uses `ZeebeClient` when it needs to start a process instance or publish a message.

We run Zeebe in single-broker mode locally, which is sufficient for course-project scale. The architecture does not require changes to move to a multi-broker cluster because the `ZEEBE_ADDRESS` environment variable is the only coupling point between the services and the engine.

0003 - Stateful Resilience and Error Handling

Status

Accepted

Context

Our platform runs long-lived, asynchronous verification and reporting processes. A `VerifyContent` instance might be suspended for minutes waiting for peer verdicts. A `ReportContent` instance might be waiting for a post owner to submit an objection within a defined window. During that time, any of the following can happen: a service container restarts due to a deployment, Zeebe's gRPC connection temporarily drops, the `user-service` is briefly unresponsive, or a developer's laptop goes to sleep.

The question is what happens to in-flight process instances and job executions when any of these events occur. If we have to design and maintain our own crash-recovery layer, writing process state to a database, checking for abandoned instances on startup, and re-enqueueing jobs, the services become significantly more complex and the risk of subtle bugs in the recovery logic is high.

Options Considered

Application-level checkpointing: each service writes its own in-flight state to a database and reconciles on startup. Pros: no hard dependency on the engine for recovery. Cons: significant extra code per service, high risk of subtle bugs in reconciliation logic, retry counter management must be built and maintained from scratch.

Zeebe's built-in job activation and retry model (chosen): the engine marks jobs as active with a configurable timeout; jobs not acknowledged within that window are automatically re-queued. Pros: zero application-level recovery code, engine-guaranteed retry semantics, crash-safe by design. Cons: Zeebe availability becomes a hard dependency for all process progression.

Human fallback via Camunda Tasklist for irreversible steps (chosen): instead of automated retry for destructive operations, failed tasks are routed to a human moderator via a User Task. Pros: prevents accidental double-execution of irreversible actions, makes oversight explicit in the BPMN model and auditable in Operate. Cons: requires someone to actively monitor the Tasklist; introduces an operational SLA concern.

Decision

We rely on Zeebe's built-in job activation and retry model as our primary resilience mechanism for transient failures. If the worker does not complete or fail the job because the service restarted, threw an unhandled exception, or lost its connection, Zeebe automatically makes the job available for the next poll cycle without any application-level intervention. Our services do not need to write their own retry or recovery logic for this class of failure; the engine handles it.

For retries on transient errors we configure the BPMN service tasks with an explicit retry count. When a worker explicitly fails a job (via a Zeebe job failure), Zeebe decrements the counter and re-activates the job after a back-off delay.

We follow the at-least-once approach described in the course material: domain state is committed before the Zeebe job is completed. This accepts the possibility of duplicate processing after recovery, but avoids silently losing a state transition. In practice, this means our resilience strategy depends on persisted workflow state and eventual consistency rather than distributed transactions.

0004 - Distributed Data Consistency: Saga Pattern over Distributed ACID Transactions

Status

Accepted

Context

Our platform spans five services, each with its own data store. The content verification flow touches at least three of them: `verification-service` validates the requester against `user-service`, issues a signature through `attestation-service`, and notifies downstream consumers via Kafka.

In a traditional relational system, changes spanning multiple tables within a single database can be wrapped in an ACID transaction. If any step fails, the whole thing rolls back atomically. In our architecture, the same guarantee is fundamentally unavailable: the services run in separate processes with separate databases, and there is no transaction coordinator that can acquire locks across all of them simultaneously.

The standard solution for this in a distributed system is a Saga — a sequence of local transactions, each committed independently, where a failure at step N triggers compensating transactions to undo steps 1 through N-1.

Options Considered

Shared database: all services write to a common PostgreSQL schema. Pros: ACID transactions available natively, trivial cross-service queries. Cons: tight schema coupling forces coordinated deployments, a single database failure takes down all services, and independent scaling becomes impractical.

Distributed transactions: a transaction coordinator acquires locks across all service databases before committing. Pros: strong consistency across services. Cons: not practically implementable across independent Python microservices; introduces a coordinator as a new single point of failure; high latency under lock contention.

Saga pattern with Zeebe orchestration (chosen): each service commits its own local transaction; compensating actions for failure scenarios are modelled explicitly as BPMN branches. Pros: no distributed locks, services remain independently deployable and evolvable, compensation logic is visible in the process model rather than scattered across event handlers. Cons: eventual consistency windows exist between service-local state and the process variables in Zeebe; compensation is semantic rather than physical rollback.

Decision

We keep a strict database-per-service model and coordinate cross-service consistency with Zeebe-based Saga orchestration. All three business flows (`RegisterUser` , `VerifyContent` , and `ReportContent`) follow the Saga approach: each service commits local state in its own boundary, and overall consistency is reached through workflow progression instead of a global ACID transaction.

At the current project stage, we do not implement explicit rollback-style compensating transactions for already committed steps. Our failure handling is based on forward recovery (retries, timeout paths, rejection/end states, and human intervention where needed).

If this system is extended toward production-grade guarantees, we can introduce explicit compensation steps for selected scenarios where business requirements demand semantic undo across services.

0005 - Message Broker Selection: Apache Kafka for Choreography Events

Status

Accepted

Context

As described in ADR-0001, we use choreography with events for side-effects. We need a broker that reliably delivers these events, can handle situations where a consumer is temporarily unavailable (without the producer having to retry), and allows us to add new consumers in the future without touching existing producers.

The two broad categories we considered were traditional message queues — represented by RabbitMQ — and log-based streaming platforms, represented by Apache Kafka.

Traditional MQ systems like RabbitMQ use a push model where the broker routes messages to registered consumers and removes them once acknowledged. They are easy to operate and well-suited to task queues. However, they are less well-suited to the pub/sub, multi-consumer fanout we need: adding a second consumer to a RabbitMQ queue means either duplicating the queue (and updating the producer to publish to both) or using an exchange — either way, there is coordination required when adding consumers. Message retention is also bounded; a message not consumed within a TTL is gone.

Kafka uses a pull model built on an append-only, partitioned log. Consumers maintain their own offsets and read at their own pace. Adding a new consumer group changes nothing for existing producers or existing consumers. Messages are retained for a configurable period regardless of whether any consumer has read them, which provides a natural buffer against consumer downtime and also makes replaying past events straightforward.

Options Considered

Direct HTTP calls from producer to each consumer: no broker required. Pros: simple, easy to reason about. Cons: producers must know all consumers; adding a new consumer requires changing producer code; no buffering for consumer downtime.

Traditional MQ (RabbitMQ): push-based, broker routes messages to registered consumers. Pros: easy to operate, low memory overhead. Cons: adding a new consumer requires binding new queues or exchanges, a configuration change that must be coordinated with the existing deployment; messages are dropped after their TTL if unconsumed.

Apache Kafka (chosen): append-only, partitioned log with consumer-managed offsets. Pros: consumers are fully independent of each other and of producers, configurable message retention survives consumer downtime, horizontal scaling via consumer groups, idempotent producer configuration prevents duplicates. Cons: significantly heavier to operate locally than a simple queue.

Decision

We use Apache Kafka as the message broker for all choreography-style event flows. The `notification-service` is a Kafka consumer using a named consumer group (`notification-service`) with manual offset commit after successful message processing.

The `verification-service` and `reporting-service` each create a `confluent-kafka` `Producer` instance at startup. The `notification-service` creates a `Consumer` that subscribes to all six notification topics in a single call. Kafka's partition-based routing

means the `notification-service` can be scaled horizontally — multiple instances can share the same consumer group and Kafka will distribute the partitions between them automatically.

0006 - Process Model Boundaries: BPMN Models and Bounded Contexts

Status

Accepted

Context

Our system is split into five bounded contexts, each with a different responsibility:

- **User Identity Context** (`user-service`): handles user onboarding and registration status.
- **Content Verification Context** (`verification-service`): runs peer-based content verification and triggers attestation for approved content.
- **Reporting Context** (`reporting-service`): handles reports, objection windows, moderation decisions, and deletion flow.
- **Attestation Context** (`attestation-service`): stores and invalidates signatures via REST APIs.
- **Notification Context** (`notification-service`): consumes Kafka events and sends/logs notifications as side effects.

The architecture question is where process boundaries should be drawn. We can either combine these flows into one large BPMN process, or keep process ownership aligned with bounded contexts. Since these contexts evolve independently and have different runtime behavior, one shared process model would create unnecessary coupling and coordination overhead.

Options Considered

Single unified BPMN model (process monolith): all flows modeled as one process definition or as sub-processes of one. Pros: single deployment artifact, shared process variables across flows. Cons: any change requires redeploying the whole model; teams across domains must coordinate releases around one shared artifact.

One BPMN model per bounded context (chosen): each domain service owns, deploys, and operates its own process definition. Pros: independent deployment cycles, clear ownership, easier scaling and troubleshooting per domain. Cons: cross-context linking must be handled through APIs/events and shared IDs rather than one global process state.

Decision

We align process boundaries with bounded context boundaries.

`RegisterUser.bpmn` is owned by the User Identity context, `VerifyContent.bpmn` by the Content Verification context, and `ReportContent.bpmn` by the Reporting context. Each service runs only the Zeebe workers for its own task types and does not execute tasks from other domains.

The Attestation and Notification contexts are intentionally not modeled as standalone BPMN processes. Attestation remains a dedicated REST capability used by orchestrated flows, and Notification remains a Kafka-driven side-effect consumer. This keeps orchestration focused on core business processes while preserving loose coupling for infrastructure-style concerns.

0007 - Asynchronous Message Correlation via Zeebe

Status

Accepted

Context

Two workflows in our system wait for external callbacks:

- `VerifyContent.bpmn` waits for a correlated peer decision message (`peer-approved` / `peer-rejected`), while `verification-service` collects individual peer verdict callbacks until that decision can be emitted.
- `ReportContent.bpmn` waits for post-owner objections.

The main problem is routing each callback to the correct waiting process instance at scale, without polling loops or manual process-instance lookups in application code.

It is also important to separate this from synchronous utility calls. `attestation-service` is used through direct REST calls inside service tasks (publish/invalidate signature). These are request/response steps in the flow, not asynchronous callback correlation points.

Options Considered

Synchronous blocking REST call: the callback handler looks up the process instance key and calls Zeebe's REST API directly to complete the waiting receive task. Pros: simple, one call per callback. Cons: tightly couples the HTTP layer to engine internals; fragile under client retries without idempotency guards; the handler must manage job activation lifecycle itself.

Application-level polling: a periodic background task checks a database for pending verdicts and resumes the process accordingly. Pros: decouples the HTTP handler from the engine. Cons: polling delay between verdict submission and process resumption; adds a database dependency; correlation logic moves out of the process model and into application code.

Zeebe native message correlation (chosen): the callback handler publishes a named Zeebe message with a correlation key; Zeebe routes it internally to the matching suspended instance. Pros: no polling, no secondary routing tables, message is buffered by the engine if the instance has not yet reached its wait state, wait state is visible in Camunda Operate. Cons: messages with no matching subscriber are silently dropped after the configured TTL.

Decision

We use Zeebe native message correlation for asynchronous callbacks and keep the waiting logic in BPMN.

In `ReportContent` , once a report is marked valid, the process enters an event-based wait: either an objection message arrives or the deadline timer fires. The objection endpoint (`POST /reports/{reportId}/objection`) publishes `post-owner-objection` with `reportId` as correlation key. In auto-object mode, the same message is published by the simulation path after a delay. This means both manual and automated objections use one correlation mechanism and resume the exact waiting instance.

In `VerifyContent` , `verification-service` collects individual peer verdict callbacks and publishes the correlated process decision message (`peer-approved` or `peer-rejected`) using `verificationId` as correlation key.

`attestation-service` remains outside this pattern by design: it is a synchronous REST utility used inside service tasks for signature create/invalidate operations, not an asynchronous callback participant.